



NYU | TANDON

PRINCIPLES OF DATABASE SYSTEMS

Project – Part II

NICE Insurance Management System

COURSE

CS-GY 6083 | Section B

SEMESTER

Spring 2026

SUBMISSION DATE

May 5, 2026

TEAM RAH

Rigved Shrivalkar | Net-ID: rss9347

Hardik Amarwani | Net-ID: ha3290

Abhishek Katke | Net-ID: ak11553

New York University

Tandon School of Engineering

TABLE OF CONTENTS

TABLE OF CONTENTS	2
Section 1: Summary	3
Section 2: Architecture	5
2.1 Technologies Used	5
2.2 Frontend Architecture	6
2.3 Backend Architecture	6
2.4 Database Architecture	6
2.4.1 Logical and Relational Diagram	7
2.5 Security Architecture	8
2.6 Workflow	8
Section 3: Database Tables and Stored Procedures	9
Section 4: DDL Scripts	12
SQL Code	12
Section 5: Demo (Screenshots)	29
Section 6: Features	33
Section 7: Extra Features	35
7.1 Docker Containerization	35
7.2 Database Indexing and Performance Optimization	35
7.3 Data Visualization and Analytics	35
7.4 Advanced Security Features	35
Section 8: Business Analysis with 6 SQL Queries	38
Section 9: Lessons Learned and Data Sources	45
What We Learned	45
Conclusion	46
Future Improvements	46
Data Sources	47

Section 1: Summary

The NICE Insurance Management System is a web-based database application developed for National Insurance Care Excellence (NICE), one of the leading insurance providers in the United States. The purpose of this project is to design and implement a centralized relational database system and an interactive web application that supports the management of Auto and Home Insurance services. The system allows customers and employees to securely manage insurance operations such as policy administration, invoice generation, premium payments, vehicle and home records, and customer account management.

The business problem addressed by this project is the lack of an integrated platform for handling multiple insurance services within a single system. This improves operational efficiency, reduces redundancy, enhances data consistency, and enables faster access to business information.

The project approach focused on applying database design principles and modern web application development techniques. The backend database was implemented using MySQL with a normalized relational schema designed through Oracle SQL Developer Data Modeler. The web application frontend and backend were developed using Flask (Python), HTML, CSS, Bootstrap, and Jinja2 templates. RESTful design principles were followed to organize application routes and business operations. The system supports CRUD operations for multiple entities including customers, home policies, auto policies, invoices, payments, vehicles, drivers, and employees.

The application improves business performance by streamlining insurance workflows and automating key operations. Customers can register, securely log in, manage policies, make payments, and track insurance details online. Employees can manage customer records, review policy information, monitor invoices, and process insurance-related activities efficiently. The centralized database design

also improves reporting and business analysis capabilities by enabling advanced SQL queries across multiple related entities.

Security was a major focus during development. Sensitive user information such as passwords is encrypted using bcrypt hashing before storage in the database. The application implements role-based access control to separate customer and employee privileges. Additional security measures include account lockout after multiple failed login attempts, CSRF protection, secure session management, login history tracking, parameterized SQL queries to prevent SQL injection attacks, and input sanitization to reduce cross-site scripting vulnerabilities.

The logical model of the system was designed to represent all major business entities and their relationships accurately. The relational model was then derived from the logical model by applying normalization rules, primary key and foreign key constraints, and business validation rules through CHECK constraints. The design includes separate entities for customers, customer types, home policies, auto policies, invoices, payments, vehicles, drivers, and intersection tables to resolve many-to-many relationships. The database schema ensures data integrity, minimizes redundancy, and supports efficient query execution.

Several assumptions were made during the database design process. Customers may own both auto and home insurance policies, which is handled through the customer type intersection table. Drivers are not necessarily policyholders and may be linked to multiple insured vehicles. Separate invoice and payment tables were created for auto and home insurance due to their independent business lifecycles. Additionally, all primary keys were implemented as surrogate numeric identifiers to maintain consistency and simplify referential integrity management.

Overall, the NICE Insurance Management System demonstrates the integration of relational database concepts, secure web development practices, and business process automation to deliver a practical and scalable insurance management solution.

Section 2: Architecture

2.1 Technologies Used

Layer	Technology Used
Frontend	HTML5, Bootstrap 5.3, Jinja2
Backend	Python Flask 3.0
Database	MySQL 8.0+
DB Connector	mysql-connector-python
Charts	Chart.js
Authentication	bcrypt password hashing
Security	CSRF tokens, parameterized SQL, role-based access

The NICE Insurance Management System follows a multi-tier web application architecture designed to provide scalability, maintainability, and secure communication between application components. The architecture consists of four major layers: Presentation Layer, Application Layer, Database Layer, and Containerization Layer. The system enables customers and employees to interact with insurance services through a browser-based interface while securely managing business operations such as policy management, invoice generation, payments, and customer administration.

The frontend interface communicates with the Flask backend server through HTTP requests. The backend processes business logic, authentication, and database transactions before interacting with the MySQL relational database. Docker containerization was used to simplify deployment and maintain a consistent development environment across systems.

Main Architectural Components

- Frontend User Interface
- Flask Backend Application
- MySQL Relational Database
- Docker Container Environment
- Authentication and Security Modules

2.2 Frontend Architecture

The frontend architecture was developed using HTML, CSS, Bootstrap, and Jinja2 templates to create a responsive and user-friendly web interface. The frontend layer is responsible for displaying application pages, collecting user input, and presenting data returned from the backend server.

Bootstrap was used to improve responsiveness and maintain a consistent user interface across different screen sizes and devices. Jinja2 templating was integrated with Flask to dynamically render web pages using database-driven content.

2.3 Backend Architecture

The backend architecture was implemented using Flask, a lightweight Python web framework. The backend layer manages application logic, request processing, user authentication, session management, database operations, and security validation.

Flask routes were designed following RESTful principles to organize CRUD operations for customers, policies, invoices, vehicles, and payments. The backend also manages user roles and authorization levels to ensure that customers and employees can only access permitted resources.

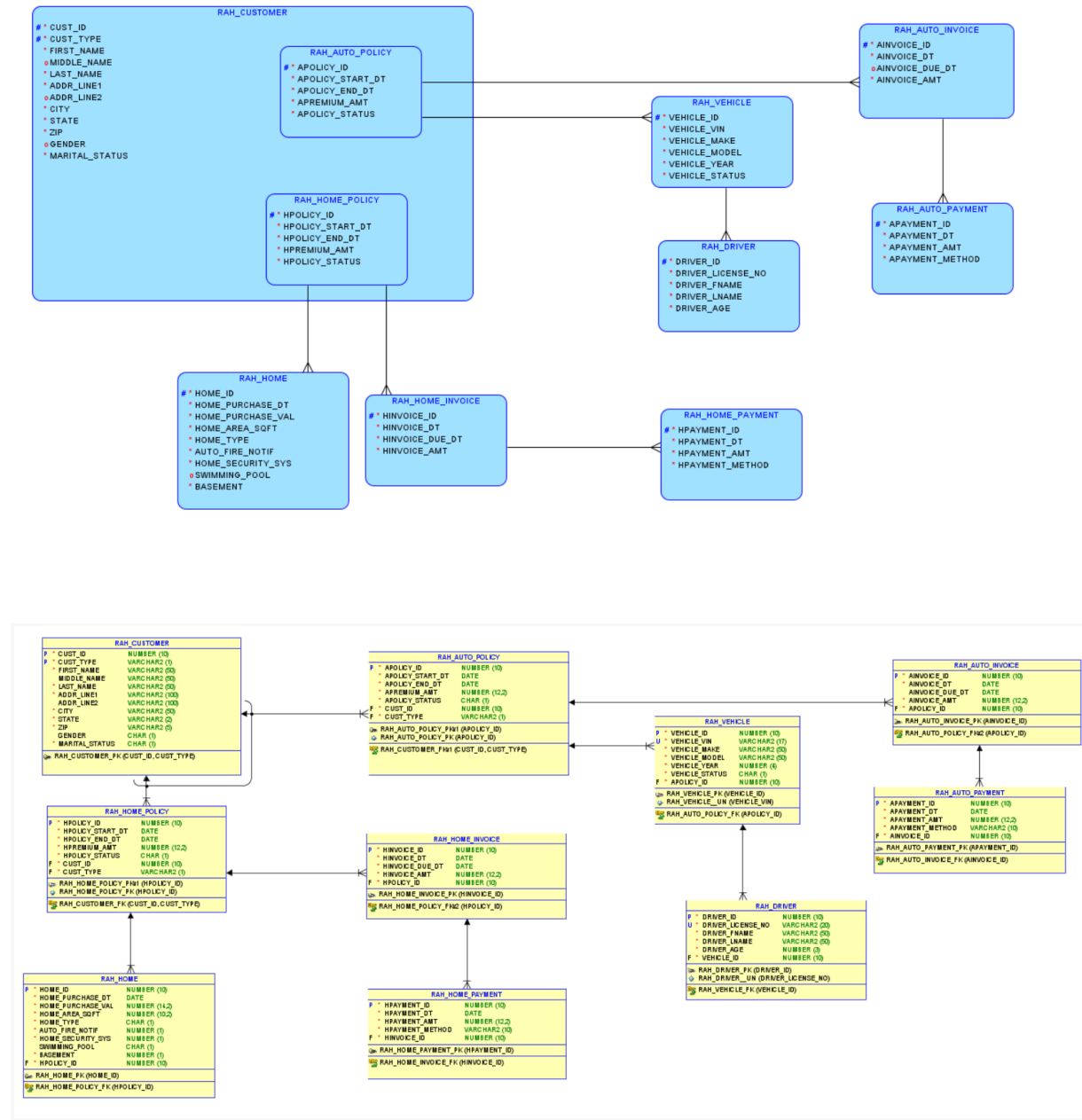
2.4 Database Architecture

The database architecture uses MySQL as the relational database management system. The database schema was designed using Oracle SQL Developer Data Modeler and follows normalization principles to reduce redundancy and improve data integrity.

The relational model contains multiple interconnected entities representing customers, insurance policies, homes, vehicles, invoices, payments, and drivers. Primary keys, foreign keys, unique constraints, and CHECK constraints were implemented to enforce business rules and maintain referential integrity.

2.4.1 Logical and Relational Diagram

Screenshot of the fully expanded Logical and Relational Model from Oracle Data Modeler is placed below. The logical model depicts all entities, their attributes, primary keys, and relationships with cardinality and participation markings



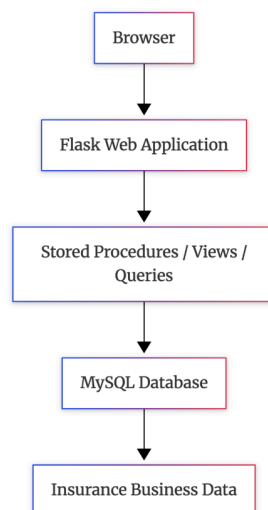
2.5 Security Architecture

Security was integrated throughout the application architecture to protect sensitive insurance and customer data. Multiple layers of security controls were implemented across the frontend, backend, and database components. Passwords are securely hashed using bcrypt before storage in the database. Parameterized SQL queries and input validation were implemented to prevent SQL Injection attacks. CSRF protection and session management mechanisms were used to improve application security and protect authenticated sessions.

These security measures help ensure confidentiality, integrity, and secure access control within the NICE Insurance Management System.

2.6 Workflow

The NICE Insurance Management System follows a request-response workflow between the client interface, backend server, and database.



Workflow Steps

1. The user accesses the application through a web browser.
2. The frontend sends HTTP requests to the Flask backend server.
3. Flask validates user authentication and authorization.
4. Business logic is processed on the backend.
5. SQL queries are executed on the MySQL database.
6. The database returns requested records or updates.
7. Flask renders the results using Jinja2 templates.
8. The frontend displays the processed response to the user.

This workflow ensures efficient communication between system components while maintaining security and data consistency.

Section 3: Database Tables and Stored Procedures

Table Name	Description	Record Count
RAH_CUSTOMER	Customer master data	20
RAH_CUST_TYPE	Customer insurance types (A/H)	27
RAH_HOME_POLICY	Home insurance policies	18
RAH_HOME	Insured home details	18
RAH_HOME_INVOICE	Home policy invoices	25
RAH_HOME_PAYMENT	Home invoice payments	20
RAH_AUTO_POLICY	Auto insurance policies	18
RAH_AUTO_INVOICE	Auto policy invoices	25
RAH_AUTO_PAYMENT	Auto invoice payments	20
RAH_VEHICLE	Insured vehicles	20
RAH_DRIVER	Registered drivers	20
RAH_VEHICLE_DRIVER	Vehicle-driver assignments (M:N)	25
RAH_USER	Application user accounts	3
RAH_LOGIN_HISTORY	Login attempt log	15
RAH_POLICY_AUDIT	Policy change audit trail	(auto-populated by triggers)
RAH_PASSWORD_RESET	Password reset tokens	0
TOTAL		274+

Stored Procedure	Feature / Purpose
sp_process_payment	Processes customer payments for home or auto invoices and records them in the correct payment table
sp_renew_policy	Renews an existing home or auto policy by expiring the old policy and creating a new active one
sp_get_customer_summary	Returns a consolidated customer summary including profile, home policies, auto policies, and financial totals
sp_generate_invoice	Creates a new invoice for either a home or auto policy
sp_register_customer	Registers a new customer account and creates the linked customer record in the database
sp_register_employee	Registers a new employee account in the system
sp_record_login_success	Logs successful login activity, resets failed attempts, and updates last login time
sp_record_login_failure	Logs failed login attempts and locks the account after repeated failures
sp_reset_password	Resets a user password and records the reset action
sp_change_address	Updates a customer's address details
sp_add_customer	Adds a new customer record for employee-side customer management
sp_update_customer	Updates an existing customer's details
sp_delete_customer	Deletes a customer record
sp_add_home_policy	Creates a new home insurance policy and its related home record
sp_add_auto_policy	Creates a new auto insurance policy
sp_update_home_policy	Updates a home policy and related home-type information

sp_update_auto_policy	Updates an auto policy's dates, premium, or status
sp_delete_home_policy	Deletes a home policy
sp_delete_auto_policy	Deletes an auto policy
sp_add_vehicle	Adds a vehicle under an auto policy
sp_update_vehicle	Updates vehicle information such as VIN, make, model, year, and ownership status
sp_delete_vehicle	Deletes a vehicle record
sp_add_driver	Adds a driver and links them to a vehicle
sp_update_driver	Updates driver details and assigned vehicle
sp_delete_driver	Deletes a driver record
sp_assign_driver	Reassigns a driver to a vehicle

Section 4: DDL Scripts

SQL Code

```
-- =====
-- NICE Insurance Database - CS-GY 6083 Part II
-- MySQL conversion of the updated Oracle Data Modeler DDL
-- Core schema: 10 business tables + 4 application extension tables
-- =====

DROP DATABASE IF EXISTS nice_insurance;
CREATE DATABASE nice_insurance CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE nice_insurance;

-- =====
-- 1. TABLE DEFINITIONS
-- =====

-- Core Table 1: RAH_CUSTOMER
CREATE TABLE RAH_CUSTOMER (
  CUST_ID INT NOT NULL COMMENT 'System-generated unique customer identifier',
  CUST_TYPE CHAR(1) NOT NULL COMMENT 'A=Auto customer, H=Home customer',
  FIRST_NAME VARCHAR(50) NOT NULL COMMENT 'Customer first name (composite resolved)',
  MIDDLE_NAME VARCHAR(50) NULL COMMENT 'Customer middle name (composite resolved)',
  LAST_NAME VARCHAR(50) NOT NULL COMMENT 'Customer last name (composite attribute resolved)',
  ADDR_LINE1 VARCHAR(100) NOT NULL COMMENT 'Street address line 1 (composite attribute resolved)',
  ADDR_LINE2 VARCHAR(100) NULL COMMENT 'Street address line 2 (composite attribute resolved)',
  CITY VARCHAR(50) NOT NULL COMMENT 'City of residence',
  STATE VARCHAR(2) NOT NULL COMMENT 'US State abbreviation',
  ZIP VARCHAR(5) NOT NULL COMMENT 'ZIP code',
  GENDER CHAR(1) NULL COMMENT 'M=Male, F=Female. Customer may choose not to provide',
  MARITAL_STATUS CHAR(1) NOT NULL COMMENT 'M=Married, S=Single, W=Widow/Widower',
  CONSTRAINT RAH_CUSTOMER_PK PRIMARY KEY (CUST_ID),
  CONSTRAINT RAH_CUSTOMER_UK_ID_TYPE UNIQUE (CUST_ID, CUST_TYPE),
  CONSTRAINT RAH_CUSTOMER_CK_3 CHECK (CUST_TYPE IN ('A','H')),
  CONSTRAINT RAH_CHK_GENDER CHECK (GENDER IN ('M','F') OR GENDER IS NULL),
  CONSTRAINT RAH_CHK_MARITAL CHECK (MARITAL_STATUS IN ('M','S','W'))
);

-- Core Table 2: RAH_HOME_POLICY
CREATE TABLE RAH_HOME_POLICY (
  HPOLICY_ID INT NOT NULL COMMENT 'Unique home insurance policy identifier',
  HPOLICY_START_DT DATE NOT NULL COMMENT 'Home insurance policy start date',
  HPOLICY_END_DT DATE NOT NULL COMMENT 'Home insurance policy end date; must be after start date',
  HPREMIUM_AMT DECIMAL(12,2) NOT NULL COMMENT 'Home insurance premium amount in USD',
  HPOLICY_STATUS CHAR(1) NOT NULL COMMENT 'C=Current, E=Expired',
  CUST_ID INT NOT NULL COMMENT 'System-generated unique customer identifier',
  CUST_TYPE CHAR(1) NOT NULL DEFAULT 'H' COMMENT 'Must be H for home policies',
  CONSTRAINT RAH_HOME_POLICY_PK PRIMARY KEY (HPOLICY_ID),
  CONSTRAINT RAH_HOME_POLICY_UK UNIQUE (HPOLICY_ID),
  CONSTRAINT RAH_CHK_HPOL_DT CHECK (HPOLICY_END_DT > HPOLICY_START_DT),
  CONSTRAINT RAH_CHK_HPREM CHECK (HPREMIUM_AMT > 0),
  CONSTRAINT RAH_CHK_HPOL_ST CHECK (HPOLICY_STATUS IN ('C','E')),
  CONSTRAINT RAH_HOME_POLICY_CK_4 CHECK (CUST_TYPE = 'H'),
  CONSTRAINT RAH_CUSTOMER_FK FOREIGN KEY (CUST_ID, CUST_TYPE)
    REFERENCES RAH_CUSTOMER (CUST_ID, CUST_TYPE) ON DELETE CASCADE
);

-- Core Table 3: RAH_HOME
CREATE TABLE RAH_HOME (
  HOME_ID INT NOT NULL COMMENT 'Unique identifier for each insured home',
  HOME_PURCHASE_DT DATE NOT NULL,
  HOME_PURCHASE_VAL DECIMAL(14,2) NOT NULL COMMENT 'Purchase value of the home in USD',
  HOME_AREA_SQFT DECIMAL(10,2) NOT NULL COMMENT 'Home area in square feet',
  HOME_TYPE CHAR(1) NOT NULL COMMENT 'S=Single Family, M=Multi Family, C=Condominium, T=Town House',
  AUTO_FIRE_NOTIF TINYINT NOT NULL COMMENT '1=Auto fire notification present, 0=Not present',
  HOME_SECURITY_SYS TINYINT NOT NULL COMMENT '1=Security system installed and monitored, 0=Not',
  SWIMMING_POOL CHAR(1) NULL COMMENT 'U=Underground, O=Overground, I=Indoor, M=Multiple, NULL=No pool',
  BASEMENT TINYINT NOT NULL COMMENT '1=Has basement, 0=No basement',
  HPOLICY_ID INT NOT NULL,
  CONSTRAINT RAH_HOME_PK PRIMARY KEY (HOME_ID),
```

```

CONSTRAINT RAH_CHK_HOMEVAL CHECK (HOME_PURCHASE_VAL > 0),
CONSTRAINT RAH_CHK_AREA CHECK (HOME_AREA_SQFT > 0),
CONSTRAINT RAH_CHK_HOME_TYP CHECK (HOME_TYPE IN ('S','M','C','T')),
CONSTRAINT RAH_CHK_FIRE CHECK (AUTO_FIRE_NOTIF IN (1,0)),
CONSTRAINT RAH_CHK_SECURITY CHECK (HOME_SECURITY_SYS IN (1,0)),
CONSTRAINT RAH_CHK_POOL CHECK (SWIMMING_POOL IN ('U','O','I','M') OR SWIMMING_POOL IS NULL),
CONSTRAINT RAH_CHK_BASEMENT CHECK (BASEMENT IN (1,0)),
CONSTRAINT RAH_HOME_POLICY_FK FOREIGN KEY (HPOLICY_ID)
    REFERENCES RAH_HOME_POLICY (HPOLICY_ID) ON DELETE CASCADE
);

```

```

-- Core Table 4: RAH_HOME_INVOICE
CREATE TABLE RAH_HOME_INVOICE (
    HINVOICE_ID INT NOT NULL COMMENT 'Unique home insurance invoice identifier',
    HINVOICE_DT DATE NOT NULL COMMENT 'Date the invoice was generated',
    HINVOICE_DUE_DT DATE NOT NULL COMMENT 'Payment due date for this invoice',
    HINVOICE_AMT DECIMAL(12,2) NOT NULL COMMENT 'Invoice amount in USD',
    HPOLICY_ID INT NOT NULL,
    CONSTRAINT RAH_HOME_INVOICE_PK PRIMARY KEY (HINVOICE_ID),
    CONSTRAINT RAH_CHK_HINV_AMT CHECK (HINVOICE_AMT > 0),
    CONSTRAINT RAH_CHK_HINV_DT CHECK (HINVOICE_DUE_DT > HINVOICE_DT),
    CONSTRAINT RAH_HOME_POLICY_FKv2 FOREIGN KEY (HPOLICY_ID)
        REFERENCES RAH_HOME_POLICY (HPOLICY_ID) ON DELETE CASCADE
);

```

```

-- Core Table 5: RAH_HOME_PAYMENT
CREATE TABLE RAH_HOME_PAYMENT (
    HPAYMENT_ID INT NOT NULL COMMENT 'Unique home insurance payment identifier',
    HPAYMENT_DT DATE NOT NULL COMMENT 'Date the payment was made',
    HPAYMENT_AMT DECIMAL(12,2) NOT NULL COMMENT 'Payment amount in USD',
    HPAYMENT_METHOD VARCHAR(10) NOT NULL COMMENT 'Method of payment accepted by NICE',
    HINVOICE_ID INT NOT NULL,
    CONSTRAINT RAH_HOME_PAYMENT_PK PRIMARY KEY (HPAYMENT_ID),
    CONSTRAINT RAH_CHK_HPAY_AMT CHECK (HPAYMENT_AMT > 0),
    CONSTRAINT RAH_CHK_HPAY_MTD CHECK (HPAYMENT_METHOD IN ('PayPal','Credit','Debit','Check')),
    CONSTRAINT RAH_HOME_INVOICE_FK FOREIGN KEY (HINVOICE_ID)
        REFERENCES RAH_HOME_INVOICE (HINVOICE_ID) ON DELETE CASCADE
);

```

```

-- Core Table 6: RAH_AUTO_POLICY
CREATE TABLE RAH_AUTO_POLICY (
    APOLICY_ID INT NOT NULL COMMENT 'Unique auto insurance policy identifier',
    APOLICY_START_DT DATE NOT NULL COMMENT 'Auto insurance policy start date',
    APOLICY_END_DT DATE NOT NULL COMMENT 'Auto insurance policy end date; must be after start date',
    APREMIUM_AMT DECIMAL(12,2) NOT NULL COMMENT 'Auto insurance premium amount in USD',
    APOLICY_STATUS CHAR(1) NOT NULL COMMENT 'C=Current, E=Expired',
    CUST_ID INT NOT NULL COMMENT 'System-generated unique customer identifier',
    CUST_TYPE CHAR(1) NOT NULL DEFAULT 'A' COMMENT 'Must be A for auto policies',
    CONSTRAINT RAH_AUTO_POLICY_PK PRIMARY KEY (APOLICY_ID),
    CONSTRAINT RAH_AUTO_POLICY_UK UNIQUE (APOLICY_ID),
    CONSTRAINT RAH_CHK_APOL_DT CHECK (APOLICY_END_DT > APOLICY_START_DT),
    CONSTRAINT RAH_CHK_APREM CHECK (APREMIUM_AMT > 0),
    CONSTRAINT RAH_CHK_APOL_ST CHECK (APOLICY_STATUS IN ('C','E')),
    CONSTRAINT RAH_AUTO_POLICY_CK_4 CHECK (CUST_TYPE = 'A'),
    CONSTRAINT RAH_CUSTOMER_FKv1 FOREIGN KEY (CUST_ID, CUST_TYPE)
        REFERENCES RAH_CUSTOMER (CUST_ID, CUST_TYPE) ON DELETE CASCADE
);

```

```

-- Core Table 7: RAH_AUTO_INVOICE
CREATE TABLE RAH_AUTO_INVOICE (
    AINVOICE_ID INT NOT NULL COMMENT 'Unique auto insurance invoice identifier',
    AINVOICE_DT DATE NOT NULL COMMENT 'Date the invoice was generated',
    AINVOICE_DUE_DT DATE NOT NULL COMMENT 'Payment due date for this invoice',
    AINVOICE_AMT DECIMAL(12,2) NOT NULL COMMENT 'Invoice amount in USD',
    APOLICY_ID INT NOT NULL,
    CONSTRAINT RAH_AUTO_INVOICE_PK PRIMARY KEY (AINVOICE_ID),
    CONSTRAINT RAH_CHK_AINV_AMT CHECK (AINVOICE_AMT > 0),
    CONSTRAINT RAH_CHK_AINV_DT CHECK (AINVOICE_DUE_DT > AINVOICE_DT),
    CONSTRAINT RAH_AUTO_POLICY_FKv2 FOREIGN KEY (APOLICY_ID)
        REFERENCES RAH_AUTO_POLICY (APOLICY_ID) ON DELETE CASCADE
);

```

```

-- Core Table 8: RAH_AUTO_PAYMENT
CREATE TABLE RAH_AUTO_PAYMENT (
    APAYMENT_ID INT NOT NULL COMMENT 'Unique auto insurance payment identifier',
    APAYMENT_DT DATE NOT NULL COMMENT 'Date the payment was made',
    APAYMENT_AMT DECIMAL(12,2) NOT NULL COMMENT 'Payment amount in USD',
    APAYMENT_METHOD VARCHAR(10) NOT NULL COMMENT 'Method of payment accepted by NICE',
    AINVOICE_ID INT NOT NULL,
    CONSTRAINT RAH_AUTO_PAYMENT_PK PRIMARY KEY (APAYMENT_ID),
    CONSTRAINT RAH_CHK_APAY_AMT CHECK (APAYMENT_AMT > 0),
    CONSTRAINT RAH_CHK_APAY_MTD CHECK (APAYMENT_METHOD IN ('PayPal','Credit','Debit','Check')),
    CONSTRAINT RAH_AUTO_INVOICE_FK FOREIGN KEY (AINVOICE_ID)

```

```
REFERENCES RAH_AUTO_INVOICE (AINVOICE_ID) ON DELETE CASCADE
);
```

-- Core Table 9: RAH_VEHICLE

```
CREATE TABLE RAH_VEHICLE (
  VEHICLE_ID INT NOT NULL COMMENT 'System-generated unique vehicle identifier',
  VEHICLE_VIN VARCHAR(17) NOT NULL COMMENT 'Vehicle Identification Number; unique per vehicle',
  VEHICLE_MAKE VARCHAR(50) NOT NULL COMMENT 'Manufacturer/make of vehicle',
  VEHICLE_MODEL VARCHAR(50) NOT NULL COMMENT 'Model of vehicle',
  VEHICLE_YEAR SMALLINT NOT NULL COMMENT 'Year of vehicle manufacture',
  VEHICLE_STATUS CHAR(1) NOT NULL COMMENT 'L=Leased, F=Financed, O=Owned',
  APOLICY_ID INT NOT NULL,
  CONSTRAINT RAH_VEHICLE_PK PRIMARY KEY (VEHICLE_ID),
  CONSTRAINT RAH_VEHICLE__UN UNIQUE (VEHICLE_VIN),
  CONSTRAINT RAH_CHK_VEH_YEAR CHECK (VEHICLE_YEAR >= 1886),
  CONSTRAINT RAH_CHK_VEH_STAT CHECK (VEHICLE_STATUS IN ('L','F','O')),
  CONSTRAINT RAH_CHK_VIN_LEN CHECK (CHAR_LENGTH(VEHICLE_VIN) = 17),
  CONSTRAINT RAH_AUTO_POLICY_FK FOREIGN KEY (APOLICY_ID)
    REFERENCES RAH_AUTO_POLICY (APOLICY_ID) ON DELETE CASCADE
);
```

-- Core Table 10: RAH_DRIVER

```
CREATE TABLE RAH_DRIVER (
  DRIVER_ID INT NOT NULL COMMENT 'System-generated unique driver identifier',
  DRIVER_LICENSE_NO VARCHAR(20) NOT NULL COMMENT 'Driver license number; unique per driver',
  DRIVER_FNAME VARCHAR(50) NOT NULL COMMENT 'Driver first name',
  DRIVER_LNAME VARCHAR(50) NOT NULL COMMENT 'Driver last name',
  DRIVER_AGE INT NOT NULL COMMENT 'Age of the driver; must be at least 16 to drive',
  VEHICLE_ID INT NOT NULL,
  CONSTRAINT RAH_DRIVER_PK PRIMARY KEY (DRIVER_ID),
  CONSTRAINT RAH_DRIVER__UN UNIQUE (DRIVER_LICENSE_NO),
  CONSTRAINT RAH_CHK_DRV_AGE CHECK (DRIVER_AGE >= 16),
  CONSTRAINT RAH_VEHICLE_FK FOREIGN KEY (VEHICLE_ID)
    REFERENCES RAH_VEHICLE (VEHICLE_ID) ON DELETE CASCADE
);
```

-- Application Table 11: RAH_USER

```
CREATE TABLE RAH_USER (
  USER_ID INT AUTO_INCREMENT PRIMARY KEY,
  USERNAME VARCHAR(50) NOT NULL UNIQUE,
  PASSWORD_HASH VARCHAR(255) NOT NULL,
  EMAIL VARCHAR(100) NOT NULL UNIQUE,
  ROLE ENUM('customer','employee') NOT NULL,
  CUST_ID INT NULL,
  SECURITY_QUESTION VARCHAR(255),
  SECURITY_ANSWER_HASH VARCHAR(255),
  FAILED_LOGIN_ATTEMPTS INT DEFAULT 0,
  ACCOUNT_LOCKED TINYINT DEFAULT 0,
  LAST_LOGIN DATETIME NULL,
  CREATED_AT DATETIME DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT RAH_USER_CUSTOMER_FK FOREIGN KEY (CUST_ID) REFERENCES RAH_CUSTOMER(CUST_ID) ON DELETE SET NULL
);
```

-- Application Table 12: RAH_LOGIN_HISTORY

```
CREATE TABLE RAH_LOGIN_HISTORY (
  LOG_ID INT AUTO_INCREMENT PRIMARY KEY,
  USER_ID INT NOT NULL,
  LOGIN_DT DATETIME NOT NULL,
  IP_ADDRESS VARCHAR(45),
  SUCCESS TINYINT NOT NULL,
  CONSTRAINT RAH_LOGIN_HISTORY_USER_FK FOREIGN KEY (USER_ID) REFERENCES RAH_USER(USER_ID) ON DELETE CASCADE
);
```

-- Application Table 13: RAH_POLICY_AUDIT

```
CREATE TABLE RAH_POLICY_AUDIT (
  AUDIT_ID INT AUTO_INCREMENT PRIMARY KEY,
  TABLE_NAME VARCHAR(50) NOT NULL,
  RECORD_ID INT NOT NULL,
  ACTION ENUM('INSERT','UPDATE','DELETE') NOT NULL,
  OLD_VALUES JSON NULL,
  NEW_VALUES JSON NULL,
  CHANGED_BY VARCHAR(50),
  CHANGED_AT DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

-- Application Table 14: RAH_PASSWORD_RESET

```
CREATE TABLE RAH_PASSWORD_RESET (
  RESET_ID INT AUTO_INCREMENT PRIMARY KEY,
  USER_ID INT NOT NULL,
  RESET_TOKEN VARCHAR(255) NOT NULL,
  EXPIRES_AT DATETIME NOT NULL,
  USED TINYINT DEFAULT 0,
  CREATED_AT DATETIME DEFAULT CURRENT_TIMESTAMP,
);
```

```
CONSTRAINT RAH_PASSWORD_RESET_USER_FK FOREIGN KEY (USER_ID) REFERENCES RAH_USER(USER_ID) ON DELETE CASCADE
);
```

```
-- =====
-- 2. STRATEGIC INDEXES (with rationale)
-- =====
```

```
-- Frequent filtering by state for reports and analytics
CREATE INDEX idx_customer_state ON RAH_CUSTOMER(STATE);

-- Frequent filtering by city for geographic analytics
CREATE INDEX idx_customer_city ON RAH_CUSTOMER(CITY);

-- Discriminator filtering for the new arc-based customer model
CREATE INDEX idx_customer_type ON RAH_CUSTOMER(CUST_TYPE);

-- Join optimization: finding all home policies for a customer
CREATE INDEX idx_home_policy_cust ON RAH_HOME_POLICY(CUST_ID, CUST_TYPE);

-- Status filtering: active vs expired policy lookups
CREATE INDEX idx_home_policy_status ON RAH_HOME_POLICY(HPOLICY_STATUS);

-- Join optimization: finding all auto policies for a customer
CREATE INDEX idx_auto_policy_cust ON RAH_AUTO_POLICY(CUST_ID, CUST_TYPE);

-- Status filtering: active vs expired auto policy lookups
CREATE INDEX idx_auto_policy_status ON RAH_AUTO_POLICY(APOLICY_STATUS);

-- Join optimization: linking invoices to home policies
CREATE INDEX idx_home_invoice_policy ON RAH_HOME_INVOICE(HPOLICY_ID);

-- Join optimization: linking invoices to auto policies
CREATE INDEX idx_auto_invoice_policy ON RAH_AUTO_INVOICE(APOLICY_ID);

-- Join optimization: finding vehicles under an auto policy
CREATE INDEX idx_vehicle_policy ON RAH_VEHICLE(APOLICY_ID);

-- Join optimization: finding drivers assigned directly to a vehicle
CREATE INDEX idx_driver_vehicle ON RAH_DRIVER(VEHICLE_ID);

-- Role-based query optimization for user authentication
CREATE INDEX idx_user_role ON RAH_USER(ROLE);

-- Customer lookup optimization when loading user profiles
CREATE INDEX idx_user_custid ON RAH_USER(CUST_ID);

-- Login history lookups by user
CREATE INDEX idx_login_history_user ON RAH_LOGIN_HISTORY(USER_ID);

-- Audit table lookups by table name and record
CREATE INDEX idx_audit_table_record ON RAH_POLICY_AUDIT(TABLE_NAME, RECORD_ID);
```

```
-- =====
-- 3. STORED PROCEDURES
-- =====
```

```
DELIMITER //
```

```
-- SP 1: Process Payment (Home or Auto)
CREATE PROCEDURE sp_process_payment(
    IN p_payment_type VARCHAR(4),
    IN p_invoice_id INT,
    IN p_amount DECIMAL(12,2),
    IN p_method VARCHAR(10),
    IN p_payment_date DATE
)
BEGIN
    DECLARE v_next_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Payment processing failed';
    END;

    START TRANSACTION;

    IF p_payment_type = 'home' THEN
        SELECT COALESCE(MAX(HPAYMENT_ID), 0) + 1 INTO v_next_id FROM RAH_HOME_PAYMENT;
        INSERT INTO RAH_HOME_PAYMENT (HPAYMENT_ID, HPAYMENT_DT, HPAYMENT_AMT, HPAYMENT_METHOD, HINVOICE_ID)
        VALUES (v_next_id, p_payment_date, p_amount, p_method, p_invoice_id);
    ELSEIF p_payment_type = 'auto' THEN
```

```

        SELECT COALESCE(MAX(APAYMENT_ID), 0) + 1 INTO v_next_id FROM RAH_AUTO_PAYMENT;
        INSERT INTO RAH_AUTO_PAYMENT (APAYMENT_ID, APAYMENT_DT, APAYMENT_AMT, APAYMENT_METHOD, AINVOICE_ID)
        VALUES (v_next_id, p_payment_date, p_amount, p_method, p_invoice_id);
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid payment type. Use home or auto.';
    END IF;

    COMMIT;
END //

-- SP 2: Renew Policy (creates new, expires old)
CREATE PROCEDURE sp_renew_policy(
    IN p_policy_type VARCHAR(4),
    IN p_old_policy_id INT,
    IN p_new_start DATE,
    IN p_new_end DATE,
    IN p_new_premium DECIMAL(12,2)
)
BEGIN
    DECLARE v_cust_id INT;
    DECLARE v_new_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Policy renewal failed';
    END;

    START TRANSACTION;

    IF p_policy_type = 'home' THEN
        SELECT CUST_ID INTO v_cust_id FROM RAH_HOME_POLICY WHERE HPOLICY_ID = p_old_policy_id;
        UPDATE RAH_HOME_POLICY SET HPOLICY_STATUS = 'E' WHERE HPOLICY_ID = p_old_policy_id;
        SELECT COALESCE(MAX(HPOLICY_ID), 0) + 1 INTO v_new_id FROM RAH_HOME_POLICY;
        INSERT INTO RAH_HOME_POLICY (HPOLICY_ID, HPOLICY_START_DT, HPOLICY_END_DT, HPREMIUM_AMT, HPOLICY_STATUS, CUST_ID,
        CUST_TYPE)
        VALUES (v_new_id, p_new_start, p_new_end, p_new_premium, 'C', v_cust_id, 'H');
    ELSEIF p_policy_type = 'auto' THEN
        SELECT CUST_ID INTO v_cust_id FROM RAH_AUTO_POLICY WHERE APOLICY_ID = p_old_policy_id;
        UPDATE RAH_AUTO_POLICY SET APOLICY_STATUS = 'E' WHERE APOLICY_ID = p_old_policy_id;
        SELECT COALESCE(MAX(APOLICY_ID), 0) + 1 INTO v_new_id FROM RAH_AUTO_POLICY;
        INSERT INTO RAH_AUTO_POLICY (APOLICY_ID, APOLICY_START_DT, APOLICY_END_DT, APREMIUM_AMT, APOLICY_STATUS, CUST_ID, CUST_TYPE)
        VALUES (v_new_id, p_new_start, p_new_end, p_new_premium, 'C', v_cust_id, 'A');
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid policy type. Use home or auto.';
    END IF;

    COMMIT;
    SELECT v_new_id AS new_policy_id;
END //

-- SP 3: Get Customer Summary
CREATE PROCEDURE sp_get_customer_summary(IN p_cust_id INT)
BEGIN
    -- Customer info
    SELECT c.*
    FROM RAH_CUSTOMER c
    WHERE c.CUST_ID = p_cust_id;

    -- Home policies
    SELECT hp.*, h.HOME_TYPE, h.HOME_PURCHASE_VAL, h.HOME_AREA_SQFT
    FROM RAH_HOME_POLICY hp
    LEFT JOIN RAH_HOME h ON hp.HPOLICY_ID = h.HPOLICY_ID
    WHERE hp.CUST_ID = p_cust_id;

    -- Auto policies
    SELECT ap.*, COUNT(v.VEHICLE_ID) AS vehicle_count
    FROM RAH_AUTO_POLICY ap
    LEFT JOIN RAH_VEHICLE v ON ap.APOLICY_ID = v.APOLICY_ID
    WHERE ap.CUST_ID = p_cust_id
    GROUP BY ap.APOLICY_ID;

    -- Financial summary
    SELECT fn_total_premium(p_cust_id) AS total_premium,
        fn_outstanding_balance(p_cust_id) AS outstanding_balance;
END //

-- SP 4: Generate Invoice
CREATE PROCEDURE sp_generate_invoice(
    IN p_policy_type VARCHAR(4),
    IN p_policy_id INT,
    IN p_invoice_date DATE,
    IN p_due_date DATE,
    IN p_amount DECIMAL(12,2)

```



```

)
BEGIN
    DECLARE v_next_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invoice generation failed';
    END;

    START TRANSACTION;

    IF p_policy_type = 'home' THEN
        SELECT COALESCE(MAX(HINVOICE_ID), 0) + 1 INTO v_next_id FROM RAH_HOME_INVOICE;
        INSERT INTO RAH_HOME_INVOICE (HINVOICE_ID, HINVOICE_DT, HINVOICE_DUE_DT, HINVOICE_AMT, HPOLICY_ID)
        VALUES (v_next_id, p_invoice_date, p_due_date, p_amount, p_policy_id);
    ELSEIF p_policy_type = 'auto' THEN
        SELECT COALESCE(MAX(AINVOICE_ID), 0) + 1 INTO v_next_id FROM RAH_AUTO_INVOICE;
        INSERT INTO RAH_AUTO_INVOICE (AINVOICE_ID, AINVOICE_DT, AINVOICE_DUE_DT, AINVOICE_AMT, APOLICY_ID)
        VALUES (v_next_id, p_invoice_date, p_due_date, p_amount, p_policy_id);
    ELSE
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Invalid policy type. Use home or auto.';
    END IF;

    COMMIT;
    SELECT v_next_id AS new_invoice_id;
END //

-- SP 5: Register a customer user and matching customer row
CREATE PROCEDURE sp_register_customer(
    IN p_username VARCHAR(50),
    IN p_password_hash VARCHAR(255),
    IN p_email VARCHAR(100),
    IN p_security_question VARCHAR(255),
    IN p_security_answer_hash VARCHAR(255),
    IN p_cust_type CHAR(1),
    IN p_first_name VARCHAR(50),
    IN p_middle_name VARCHAR(50),
    IN p_last_name VARCHAR(50),
    IN p_addr_line1 VARCHAR(100),
    IN p_addr_line2 VARCHAR(100),
    IN p_city VARCHAR(50),
    IN p_state VARCHAR(2),
    IN p_zip VARCHAR(5),
    IN p_gender CHAR(1),
    IN p_marital_status CHAR(1)
)
BEGIN
    DECLARE v_new_cust_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Customer registration failed';
    END;

    START TRANSACTION;
    SELECT COALESCE(MAX(CUST_ID), 0) + 1 INTO v_new_cust_id FROM RAH_CUSTOMER;
    INSERT INTO RAH_CUSTOMER (
        CUST_ID, CUST_TYPE, FIRST_NAME, MIDDLE_NAME, LAST_NAME,
        ADDR_LINE1, ADDR_LINE2, CITY, STATE, ZIP, GENDER, MARITAL_STATUS
    )
    VALUES (
        v_new_cust_id, p_cust_type, p_first_name, p_middle_name, p_last_name,
        p_addr_line1, p_addr_line2, p_city, p_state, p_zip, p_gender, p_marital_status
    );
    INSERT INTO RAH_USER (
        USERNAME, PASSWORD_HASH, EMAIL, ROLE, CUST_ID, SECURITY_QUESTION, SECURITY_ANSWER_HASH
    )
    VALUES (
        p_username, p_password_hash, p_email, 'customer', v_new_cust_id,
        p_security_question, p_security_answer_hash
    );
    COMMIT;
    SELECT v_new_cust_id AS new_cust_id;
END //

CREATE PROCEDURE sp_register_employee(
    IN p_username VARCHAR(50),
    IN p_password_hash VARCHAR(255),
    IN p_email VARCHAR(100),
    IN p_security_question VARCHAR(255),
    IN p_security_answer_hash VARCHAR(255)
)
BEGIN

```

```

DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Employee registration failed';
END;

START TRANSACTION;
INSERT INTO RAH_USER (
    USERNAME, PASSWORD_HASH, EMAIL, ROLE, CUST_ID, SECURITY_QUESTION, SECURITY_ANSWER_HASH
)
VALUES (
    p_username, p_password_hash, p_email, 'employee', NULL,
    p_security_question, p_security_answer_hash
);
COMMIT;
END //

CREATE PROCEDURE sp_record_login_success(IN p_user_id INT, IN p_ip_address VARCHAR(45))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Login success recording failed';
    END;

    START TRANSACTION;
    UPDATE RAH_USER
    SET FAILED_LOGIN_ATTEMPTS = 0,
        LAST_LOGIN = NOW()
    WHERE USER_ID = p_user_id;
    INSERT INTO RAH_LOGIN_HISTORY (USER_ID, LOGIN_DT, IP_ADDRESS, SUCCESS)
    VALUES (p_user_id, NOW(), p_ip_address, 1);
    COMMIT;
END //

CREATE PROCEDURE sp_record_login_failure(IN p_user_id INT, IN p_ip_address VARCHAR(45))
BEGIN
    DECLARE v_attempts INT;
    DECLARE v_locked TINYINT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Login failure recording failed';
    END;

    START TRANSACTION;
    SELECT FAILED_LOGIN_ATTEMPTS + 1 INTO v_attempts
    FROM RAH_USER
    WHERE USER_ID = p_user_id
    FOR UPDATE;
    SET v_locked = IF(v_attempts >= 5, 1, 0);
    UPDATE RAH_USER
    SET FAILED_LOGIN_ATTEMPTS = v_attempts,
        ACCOUNT_LOCKED = v_locked
    WHERE USER_ID = p_user_id;
    INSERT INTO RAH_LOGIN_HISTORY (USER_ID, LOGIN_DT, IP_ADDRESS, SUCCESS)
    VALUES (p_user_id, NOW(), p_ip_address, 0);
    COMMIT;
    SELECT v_attempts AS failed_attempts, v_locked AS account_locked;
END //

CREATE PROCEDURE sp_reset_password(IN p_user_id INT, IN p_new_hash VARCHAR(255), IN p_token VARCHAR(255))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Password reset failed';
    END;

    START TRANSACTION;
    UPDATE RAH_USER
    SET PASSWORD_HASH = p_new_hash,
        FAILED_LOGIN_ATTEMPTS = 0,
        ACCOUNT_LOCKED = 0
    WHERE USER_ID = p_user_id;
    INSERT INTO RAH_PASSWORD_RESET (USER_ID, RESET_TOKEN, EXPIRES_AT, USED)
    VALUES (p_user_id, p_token, NOW(), 1);
    COMMIT;
END //

CREATE PROCEDURE sp_change_address(
    IN p_cust_id INT,
    IN p_addr_line1 VARCHAR(100),

```

```

    IN p_addr_line2 VARCHAR(100),
    IN p_city VARCHAR(50),
    IN p_state VARCHAR(2),
    IN p_zip VARCHAR(5)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Change address failed';
    END;

    START TRANSACTION;
    UPDATE RAH_CUSTOMER
    SET ADDR_LINE1 = p_addr_line1,
        ADDR_LINE2 = p_addr_line2,
        CITY = p_city,
        STATE = p_state,
        ZIP = p_zip
    WHERE CUST_ID = p_cust_id;
    COMMIT;
END //

CREATE PROCEDURE sp_add_customer(
    IN p_cust_type CHAR(1),
    IN p_first_name VARCHAR(50),
    IN p_middle_name VARCHAR(50),
    IN p_last_name VARCHAR(50),
    IN p_addr_line1 VARCHAR(100),
    IN p_addr_line2 VARCHAR(100),
    IN p_city VARCHAR(50),
    IN p_state VARCHAR(2),
    IN p_zip VARCHAR(5),
    IN p_gender CHAR(1),
    IN p_marital_status CHAR(1)
)
BEGIN
    DECLARE v_new_cust_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Customer creation failed';
    END;

    START TRANSACTION;
    SELECT COALESCE(MAX(CUST_ID), 0) + 1 INTO v_new_cust_id FROM RAH_CUSTOMER;
    INSERT INTO RAH_CUSTOMER (
        CUST_ID, CUST_TYPE, FIRST_NAME, MIDDLE_NAME, LAST_NAME,
        ADDR_LINE1, ADDR_LINE2, CITY, STATE, ZIP, GENDER, MARITAL_STATUS
    )
    VALUES (
        v_new_cust_id, p_cust_type, p_first_name, p_middle_name, p_last_name,
        p_addr_line1, p_addr_line2, p_city, p_state, p_zip, p_gender, p_marital_status
    );
    COMMIT;
    SELECT v_new_cust_id AS new_cust_id;
END //

CREATE PROCEDURE sp_update_customer(
    IN p_cust_id INT,
    IN p_cust_type CHAR(1),
    IN p_first_name VARCHAR(50),
    IN p_middle_name VARCHAR(50),
    IN p_last_name VARCHAR(50),
    IN p_addr_line1 VARCHAR(100),
    IN p_addr_line2 VARCHAR(100),
    IN p_city VARCHAR(50),
    IN p_state VARCHAR(2),
    IN p_zip VARCHAR(5),
    IN p_gender CHAR(1),
    IN p_marital_status CHAR(1)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Customer update failed';
    END;

    START TRANSACTION;
    UPDATE RAH_CUSTOMER
    SET CUST_TYPE = p_cust_type,
        FIRST_NAME = p_first_name,

```

```

        MIDDLE_NAME = p_middle_name,
        LAST_NAME = p_last_name,
        ADDR_LINE1 = p_addr_line1,
        ADDR_LINE2 = p_addr_line2,
        CITY = p_city,
        STATE = p_state,
        ZIP = p_zip,
        GENDER = p_gender,
        MARITAL_STATUS = p_marital_status
    WHERE CUST_ID = p_cust_id;
    COMMIT;
END //

CREATE PROCEDURE sp_delete_customer(IN p_cust_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Customer deletion failed';
    END;

    START TRANSACTION;
    DELETE FROM RAH_CUSTOMER WHERE CUST_ID = p_cust_id;
    COMMIT;
END //

CREATE PROCEDURE sp_add_home_policy(
    IN p_cust_id INT,
    IN p_start_dt DATE,
    IN p_end_dt DATE,
    IN p_premium DECIMAL(12,2),
    IN p_status CHAR(1),
    IN p_home_type CHAR(1)
)
BEGIN
    DECLARE v_policy_id INT;
    DECLARE v_home_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Home policy creation failed';
    END;

    START TRANSACTION;
    SELECT COALESCE(MAX(HPOLICY_ID), 0) + 1 INTO v_policy_id FROM RAH_HOME_POLICY;
    INSERT INTO RAH_HOME_POLICY (
        HPOLICY_ID, HPOLICY_START_DT, HPOLICY_END_DT, HPREMIUM_AMT, HPOLICY_STATUS, CUST_ID, CUST_TYPE
    )
    VALUES (v_policy_id, p_start_dt, p_end_dt, p_premium, p_status, p_cust_id, 'H');

    SELECT COALESCE(MAX(HOME_ID), 0) + 1 INTO v_home_id FROM RAH_HOME;
    INSERT INTO RAH_HOME (
        HOME_ID, HOME_PURCHASE_DT, HOME_PURCHASE_VAL, HOME_AREA_SQFT,
        HOME_TYPE, AUTO_FIRE_NOTIF, HOME_SECURITY_SYS, SWIMMING_POOL, BASEMENT, HPOLICY_ID
    )
    VALUES (v_home_id, p_start_dt, 250000.00, 1500.00, p_home_type, 0, 0, NULL, 0, v_policy_id);
    COMMIT;
    SELECT v_policy_id AS new_policy_id;
END //

CREATE PROCEDURE sp_add_auto_policy(
    IN p_cust_id INT,
    IN p_start_dt DATE,
    IN p_end_dt DATE,
    IN p_premium DECIMAL(12,2),
    IN p_status CHAR(1)
)
BEGIN
    DECLARE v_policy_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto policy creation failed';
    END;

    START TRANSACTION;
    SELECT COALESCE(MAX(APOLICY_ID), 0) + 1 INTO v_policy_id FROM RAH_AUTO_POLICY;
    INSERT INTO RAH_AUTO_POLICY (
        APOLICY_ID, APOLICY_START_DT, APOLICY_END_DT, APREMIUM_AMT, APOLICY_STATUS, CUST_ID, CUST_TYPE
    )
    VALUES (v_policy_id, p_start_dt, p_end_dt, p_premium, p_status, p_cust_id, 'A');
    COMMIT;
    SELECT v_policy_id AS new_policy_id;
END //

```

```

END //

CREATE PROCEDURE sp_update_home_policy(
    IN p_policy_id INT,
    IN p_start_dt DATE,
    IN p_end_dt DATE,
    IN p_premium DECIMAL(12,2),
    IN p_status CHAR(1),
    IN p_home_type CHAR(1)
)
BEGIN
    DECLARE v_home_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Home policy update failed';
    END;

    START TRANSACTION;
    UPDATE RAH_HOME_POLICY
    SET HPOLICY_START_DT = p_start_dt,
        HPOLICY_END_DT = p_end_dt,
        HPREMIUM_AMT = p_premium,
        HPOLICY_STATUS = p_status
    WHERE HPOLICY_ID = p_policy_id;

    SELECT MIN(HOME_ID) INTO v_home_id
    FROM RAH_HOME
    WHERE HPOLICY_ID = p_policy_id;

    IF v_home_id IS NULL THEN
        SELECT COALESCE(MAX(HOME_ID), 0) + 1 INTO v_home_id FROM RAH_HOME;
        INSERT INTO RAH_HOME (
            HOME_ID, HOME_PURCHASE_DT, HOME_PURCHASE_VAL, HOME_AREA_SQFT,
            HOME_TYPE, AUTO_FIRE_NOTIF, HOME_SECURITY_SYS, SWIMMING_POOL, BASEMENT, HPOLICY_ID
        )
        VALUES (v_home_id, p_start_dt, 250000.00, 1500.00, p_home_type, 0, 0, NULL, 0, p_policy_id);
    ELSE
        UPDATE RAH_HOME SET HOME_TYPE = p_home_type WHERE HOME_ID = v_home_id;
    END IF;
    COMMIT;
END //

CREATE PROCEDURE sp_update_auto_policy(
    IN p_policy_id INT,
    IN p_start_dt DATE,
    IN p_end_dt DATE,
    IN p_premium DECIMAL(12,2),
    IN p_status CHAR(1)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto policy update failed';
    END;

    START TRANSACTION;
    UPDATE RAH_AUTO_POLICY
    SET APOLICY_START_DT = p_start_dt,
        APOLICY_END_DT = p_end_dt,
        APREMIUM_AMT = p_premium,
        APOLICY_STATUS = p_status
    WHERE APOLICY_ID = p_policy_id;
    COMMIT;
END //

CREATE PROCEDURE sp_delete_home_policy(IN p_policy_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Home policy deletion failed';
    END;

    START TRANSACTION;
    DELETE FROM RAH_HOME_POLICY WHERE HPOLICY_ID = p_policy_id;
    COMMIT;
END //

CREATE PROCEDURE sp_delete_auto_policy(IN p_policy_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION

```

```

BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto policy deletion failed';
END;

START TRANSACTION;
DELETE FROM RAH_AUTO_POLICY WHERE APOLICY_ID = p_policy_id;
COMMIT;
END //

CREATE PROCEDURE sp_add_vehicle(
    IN p_vin VARCHAR(17),
    IN p_make VARCHAR(50),
    IN p_model VARCHAR(50),
    IN p_year SMALLINT,
    IN p_status CHAR(1),
    IN p_apolicy_id INT
)
BEGIN
    DECLARE v_vehicle_id INT;
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle creation failed';
    END;

    START TRANSACTION;
    SELECT COALESCE(MAX(VEHICLE_ID), 0) + 1 INTO v_vehicle_id FROM RAH_VEHICLE;
    INSERT INTO RAH_VEHICLE (
        VEHICLE_ID, VEHICLE_VIN, VEHICLE_MAKE, VEHICLE_MODEL, VEHICLE_YEAR, VEHICLE_STATUS, APOLICY_ID
    )
    VALUES (v_vehicle_id, p_vin, p_make, p_model, p_year, p_status, p_apolicy_id);
    COMMIT;
    SELECT v_vehicle_id AS new_vehicle_id;
END //

CREATE PROCEDURE sp_update_vehicle(
    IN p_vehicle_id INT,
    IN p_vin VARCHAR(17),
    IN p_make VARCHAR(50),
    IN p_model VARCHAR(50),
    IN p_year SMALLINT,
    IN p_status CHAR(1)
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle update failed';
    END;

    START TRANSACTION;
    UPDATE RAH_VEHICLE
    SET VEHICLE_VIN = p_vin,
        VEHICLE_MAKE = p_make,
        VEHICLE_MODEL = p_model,
        VEHICLE_YEAR = p_year,
        VEHICLE_STATUS = p_status
    WHERE VEHICLE_ID = p_vehicle_id;
    COMMIT;
END //

CREATE PROCEDURE sp_delete_vehicle(IN p_vehicle_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle deletion failed';
    END;

    START TRANSACTION;
    DELETE FROM RAH_VEHICLE WHERE VEHICLE_ID = p_vehicle_id;
    COMMIT;
END //

CREATE PROCEDURE sp_add_driver(
    IN p_license_no VARCHAR(20),
    IN p_fname VARCHAR(50),
    IN p_lname VARCHAR(50),
    IN p_age INT,
    IN p_vehicle_id INT
)
BEGIN

```

```

DECLARE v_driver_id INT;
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver creation failed';
END;

START TRANSACTION;
SELECT COALESCE(MAX(DRIVER_ID), 0) + 1 INTO v_driver_id FROM RAH_DRIVER;
INSERT INTO RAH_DRIVER (
    DRIVER_ID, DRIVER_LICENSE_NO, DRIVER_FNAME, DRIVER_LNAME, DRIVER_AGE, VEHICLE_ID
)
VALUES (v_driver_id, p_license_no, p_fname, p_lname, p_age, p_vehicle_id);
COMMIT;
SELECT v_driver_id AS new_driver_id;
END //

```

```

CREATE PROCEDURE sp_update_driver(
    IN p_driver_id INT,
    IN p_license_no VARCHAR(20),
    IN p_fname VARCHAR(50),
    IN p_lname VARCHAR(50),
    IN p_age INT,
    IN p_vehicle_id INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver update failed';
    END;

    START TRANSACTION;
    UPDATE RAH_DRIVER
    SET DRIVER_LICENSE_NO = p_license_no,
        DRIVER_FNAME = p_fname,
        DRIVER_LNAME = p_lname,
        DRIVER_AGE = p_age,
        VEHICLE_ID = p_vehicle_id
    WHERE DRIVER_ID = p_driver_id;
    COMMIT;
END //

```

```

CREATE PROCEDURE sp_delete_driver(IN p_driver_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver deletion failed';
    END;

    START TRANSACTION;
    DELETE FROM RAH_DRIVER WHERE DRIVER_ID = p_driver_id;
    COMMIT;
END //

```

```

CREATE PROCEDURE sp_assign_driver(IN p_driver_id INT, IN p_vehicle_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver assignment failed';
    END;

    START TRANSACTION;
    UPDATE RAH_DRIVER SET VEHICLE_ID = p_vehicle_id WHERE DRIVER_ID = p_driver_id;
    COMMIT;
END //

```

```

CREATE PROCEDURE sp_assign_driver(IN p_driver_id INT, IN p_vehicle_id INT)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Driver assignment failed';
    END;

    START TRANSACTION;
    UPDATE RAH_DRIVER SET VEHICLE_ID = p_vehicle_id WHERE DRIVER_ID = p_driver_id;
    COMMIT;
END //

```

```

-- =====
-- 4. USER-DEFINED FUNCTIONS
-- =====

```

```

-- Function 1: Total Premium for a Customer
CREATE FUNCTION fn_total_premium(p_cust_id INT)
RETURNS DECIMAL(14,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_home_total DECIMAL(14,2);
    DECLARE v_auto_total DECIMAL(14,2);

```

```

SELECT COALESCE(SUM(HPREMIUM_AMT), 0) INTO v_home_total
FROM RAH_HOME_POLICY WHERE CUST_ID = p_cust_id AND HPOLICY_STATUS = 'C';

SELECT COALESCE(SUM(APREMIUM_AMT), 0) INTO v_auto_total
FROM RAH_AUTO_POLICY WHERE CUST_ID = p_cust_id AND APOLICY_STATUS = 'C';

RETURN v_home_total + v_auto_total;
END //

-- Function 2: Outstanding Balance for a Customer
CREATE FUNCTION fn_outstanding_balance(p_cust_id INT)
RETURNS DECIMAL(14,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_home_invoiced DECIMAL(14,2);
    DECLARE v_home_paid DECIMAL(14,2);
    DECLARE v_auto_invoiced DECIMAL(14,2);
    DECLARE v_auto_paid DECIMAL(14,2);

    SELECT COALESCE(SUM(hi.HINVOICE_AMT), 0) INTO v_home_invoiced
    FROM RAH_HOME_INVOICE hi
    JOIN RAH_HOME_POLICY hp ON hi.HPOLICY_ID = hp.HPOLICY_ID
    WHERE hp.CUST_ID = p_cust_id;

    SELECT COALESCE(SUM(hpay.HPAYMENT_AMT), 0) INTO v_home_paid
    FROM RAH_HOME_PAYMENT hpay
    JOIN RAH_HOME_INVOICE hi ON hpay.HINVOICE_ID = hi.HINVOICE_ID
    JOIN RAH_HOME_POLICY hp ON hi.HPOLICY_ID = hp.HPOLICY_ID
    WHERE hp.CUST_ID = p_cust_id;

    SELECT COALESCE(SUM(ai.AINVOICE_AMT), 0) INTO v_auto_invoiced
    FROM RAH_AUTO_INVOICE ai
    JOIN RAH_AUTO_POLICY ap ON ai.APOLICY_ID = ap.APOLICY_ID
    WHERE ap.CUST_ID = p_cust_id;

    SELECT COALESCE(SUM(apay.APAYMENT_AMT), 0) INTO v_auto_paid
    FROM RAH_AUTO_PAYMENT apay
    JOIN RAH_AUTO_INVOICE ai ON apay.AINVOICE_ID = ai.AINVOICE_ID
    JOIN RAH_AUTO_POLICY ap ON ai.APOLICY_ID = ap.APOLICY_ID
    WHERE ap.CUST_ID = p_cust_id;

    RETURN (v_home_invoiced + v_auto_invoiced) - (v_home_paid + v_auto_paid);
END //

-- =====
-- 5. TRIGGERS (Audit Trail)
-- =====

-- Oracle arc triggers converted to MySQL SIGNAL logic.
CREATE TRIGGER ARC_FKArc_1_RAH_HOME_POLICY_BI
BEFORE INSERT ON RAH_HOME_POLICY
FOR EACH ROW
BEGIN
    IF NEW.CUST_TYPE <> 'H' OR NOT EXISTS (
        SELECT 1 FROM RAH_CUSTOMER c WHERE c.CUST_ID = NEW.CUST_ID AND c.CUST_TYPE = 'H'
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Home policy customer must have CUST_TYPE H';
    END IF;
END //

CREATE TRIGGER ARC_FKArc_1_RAH_HOME_POLICY_BU
BEFORE UPDATE ON RAH_HOME_POLICY
FOR EACH ROW
BEGIN
    IF NEW.CUST_TYPE <> 'H' OR NOT EXISTS (
        SELECT 1 FROM RAH_CUSTOMER c WHERE c.CUST_ID = NEW.CUST_ID AND c.CUST_TYPE = 'H'
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Home policy customer must have CUST_TYPE H';
    END IF;
END //

CREATE TRIGGER ARC_FKArc_1_RAH_AUTO_POLICY_BI
BEFORE INSERT ON RAH_AUTO_POLICY
FOR EACH ROW
BEGIN
    IF NEW.CUST_TYPE <> 'A' OR NOT EXISTS (
        SELECT 1 FROM RAH_CUSTOMER c WHERE c.CUST_ID = NEW.CUST_ID AND c.CUST_TYPE = 'A'
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto policy customer must have CUST_TYPE A';
    END IF;
END //

```



```

CREATE TRIGGER ARC_FKArc_1_RAH_AUTO_POLICY_BU
BEFORE UPDATE ON RAH_AUTO_POLICY
FOR EACH ROW
BEGIN
    IF NEW.CUST_TYPE <> 'A' OR NOT EXISTS (
        SELECT 1 FROM RAH_CUSTOMER c WHERE c.CUST_ID = NEW.CUST_ID AND c.CUST_TYPE = 'A'
    ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Auto policy customer must have CUST_TYPE A';
    END IF;
END //

-- Home Policy Audit Triggers
CREATE TRIGGER trg_home_policy_audit_insert
AFTER INSERT ON RAH_HOME_POLICY
FOR EACH ROW
BEGIN
    INSERT INTO RAH_POLICY_AUDIT (TABLE_NAME, RECORD_ID, ACTION, OLD_VALUES, NEW_VALUES, CHANGED_BY, CHANGED_AT)
    VALUES ('RAH_HOME_POLICY', NEW.HPOLICY_ID, 'INSERT', NULL,
        JSON_OBJECT('HPOLICY_ID', NEW.HPOLICY_ID, 'HPOLICY_START_DT', NEW.HPOLICY_START_DT,
            'HPOLICY_END_DT', NEW.HPOLICY_END_DT, 'HPREMIUM_AMT', NEW.HPREMIUM_AMT,
            'HPOLICY_STATUS', NEW.HPOLICY_STATUS, 'CUST_ID', NEW.CUST_ID),
        CURRENT_USER(), NOW());
END //

CREATE TRIGGER trg_home_policy_audit_update
AFTER UPDATE ON RAH_HOME_POLICY
FOR EACH ROW
BEGIN
    INSERT INTO RAH_POLICY_AUDIT (TABLE_NAME, RECORD_ID, ACTION, OLD_VALUES, NEW_VALUES, CHANGED_BY, CHANGED_AT)
    VALUES ('RAH_HOME_POLICY', NEW.HPOLICY_ID, 'UPDATE',
        JSON_OBJECT('HPOLICY_ID', OLD.HPOLICY_ID, 'HPOLICY_START_DT', OLD.HPOLICY_START_DT,
            'HPOLICY_END_DT', OLD.HPOLICY_END_DT, 'HPREMIUM_AMT', OLD.HPREMIUM_AMT,
            'HPOLICY_STATUS', OLD.HPOLICY_STATUS, 'CUST_ID', OLD.CUST_ID),
        JSON_OBJECT('HPOLICY_ID', NEW.HPOLICY_ID, 'HPOLICY_START_DT', NEW.HPOLICY_START_DT,
            'HPOLICY_END_DT', NEW.HPOLICY_END_DT, 'HPREMIUM_AMT', NEW.HPREMIUM_AMT,
            'HPOLICY_STATUS', NEW.HPOLICY_STATUS, 'CUST_ID', NEW.CUST_ID),
        CURRENT_USER(), NOW());
END //

CREATE TRIGGER trg_home_policy_audit_delete
BEFORE DELETE ON RAH_HOME_POLICY
FOR EACH ROW
BEGIN
    INSERT INTO RAH_POLICY_AUDIT (TABLE_NAME, RECORD_ID, ACTION, OLD_VALUES, NEW_VALUES, CHANGED_BY, CHANGED_AT)
    VALUES ('RAH_HOME_POLICY', OLD.HPOLICY_ID, 'DELETE',
        JSON_OBJECT('HPOLICY_ID', OLD.HPOLICY_ID, 'HPOLICY_START_DT', OLD.HPOLICY_START_DT,
            'HPOLICY_END_DT', OLD.HPOLICY_END_DT, 'HPREMIUM_AMT', OLD.HPREMIUM_AMT,
            'HPOLICY_STATUS', OLD.HPOLICY_STATUS, 'CUST_ID', OLD.CUST_ID),
        NULL, CURRENT_USER(), NOW());
END //

-- Auto Policy Audit Triggers
CREATE TRIGGER trg_auto_policy_audit_insert
AFTER INSERT ON RAH_AUTO_POLICY
FOR EACH ROW
BEGIN
    INSERT INTO RAH_POLICY_AUDIT (TABLE_NAME, RECORD_ID, ACTION, OLD_VALUES, NEW_VALUES, CHANGED_BY, CHANGED_AT)
    VALUES ('RAH_AUTO_POLICY', NEW.APOLICY_ID, 'INSERT', NULL,
        JSON_OBJECT('APOLICY_ID', NEW.APOLICY_ID, 'APOLICY_START_DT', NEW.APOLICY_START_DT,
            'APOLICY_END_DT', NEW.APOLICY_END_DT, 'APREMIUM_AMT', NEW.APREMIUM_AMT,
            'APOLICY_STATUS', NEW.APOLICY_STATUS, 'CUST_ID', NEW.CUST_ID),
        CURRENT_USER(), NOW());
END //

CREATE TRIGGER trg_auto_policy_audit_update
AFTER UPDATE ON RAH_AUTO_POLICY
FOR EACH ROW
BEGIN
    INSERT INTO RAH_POLICY_AUDIT (TABLE_NAME, RECORD_ID, ACTION, OLD_VALUES, NEW_VALUES, CHANGED_BY, CHANGED_AT)
    VALUES ('RAH_AUTO_POLICY', NEW.APOLICY_ID, 'UPDATE',
        JSON_OBJECT('APOLICY_ID', OLD.APOLICY_ID, 'APOLICY_START_DT', OLD.APOLICY_START_DT,
            'APOLICY_END_DT', OLD.APOLICY_END_DT, 'APREMIUM_AMT', OLD.APREMIUM_AMT,
            'APOLICY_STATUS', OLD.APOLICY_STATUS, 'CUST_ID', OLD.CUST_ID),
        JSON_OBJECT('APOLICY_ID', NEW.APOLICY_ID, 'APOLICY_START_DT', NEW.APOLICY_START_DT,
            'APOLICY_END_DT', NEW.APOLICY_END_DT, 'APREMIUM_AMT', NEW.APREMIUM_AMT,
            'APOLICY_STATUS', NEW.APOLICY_STATUS, 'CUST_ID', NEW.CUST_ID),
        CURRENT_USER(), NOW());
END //

CREATE TRIGGER trg_auto_policy_audit_delete
BEFORE DELETE ON RAH_AUTO_POLICY
FOR EACH ROW

```

```

BEGIN
    INSERT INTO RAH_POLICY_AUDIT (TABLE_NAME, RECORD_ID, ACTION, OLD_VALUES, NEW_VALUES, CHANGED_BY, CHANGED_AT)
    VALUES ('RAH_AUTO_POLICY', OLD.APOLICY_ID, 'DELETE',
        JSON_OBJECT('APOLICY_ID', OLD.APOLICY_ID, 'APOLICY_START_DT', OLD.APOLICY_START_DT,
            'APOLICY_END_DT', OLD.APOLICY_END_DT, 'APREMIUM_AMT', OLD.APREMIUM_AMT,
            'APOLICY_STATUS', OLD.APOLICY_STATUS, 'CUST_ID', OLD.CUST_ID),
        NULL, CURRENT_USER(), NOW());
END //

```

```

DELIMITER ;

```

```

-- =====
-- 7. READ-ONLY SECURITY VIEWS
-- =====

```

```

CREATE VIEW vw_customer_directory AS
SELECT
    c.CUST_ID AS customer_ref,
    c.CUST_TYPE AS customer_kind,
    c.FIRST_NAME,
    c.MIDDLE_NAME,
    c.LAST_NAME,
    c.ADDR_LINE1,
    c.ADDR_LINE2,
    c.CITY,
    c.STATE,
    c.ZIP,
    c.GENDER,
    c.MARITAL_STATUS,
    COUNT(*) AS row_marker
FROM RAH_CUSTOMER c
GROUP BY c.CUST_ID, c.CUST_TYPE, c.FIRST_NAME, c.MIDDLE_NAME, c.LAST_NAME,
    c.ADDR_LINE1, c.ADDR_LINE2, c.CITY, c.STATE, c.ZIP, c.GENDER, c.MARITAL_STATUS;

```

```

CREATE VIEW vw_home_policies AS
SELECT
    hp.HPOLICY_ID AS policy_ref,
    hp.CUST_ID AS customer_ref,
    c.FIRST_NAME,
    c.LAST_NAME,
    hp.HPOLICY_START_DT AS start_dt,
    hp.HPOLICY_END_DT AS end_dt,
    hp.HPREMIUM_AMT AS premium,
    hp.HPOLICY_STATUS AS status_code,
    h.HOME_TYPE,
    h.HOME_PURCHASE_VAL,
    h.HOME_AREA_SQFT,
    h.AUTO_FIRE_NOTIF,
    h.HOME_SECURITY_SYS,
    h.SWIMMING_POOL,
    h.BASEMENT,
    COUNT(h.HOME_ID) AS home_count
FROM RAH_HOME_POLICY hp
JOIN RAH_CUSTOMER c ON hp.CUST_ID = c.CUST_ID AND hp.CUST_TYPE = c.CUST_TYPE
LEFT JOIN RAH_HOME h ON hp.HPOLICY_ID = h.HPOLICY_ID
GROUP BY hp.HPOLICY_ID, hp.CUST_ID, c.FIRST_NAME, c.LAST_NAME,
    hp.HPOLICY_START_DT, hp.HPOLICY_END_DT, hp.HPREMIUM_AMT, hp.HPOLICY_STATUS,
    h.HOME_ID, h.HOME_TYPE, h.HOME_PURCHASE_VAL, h.HOME_AREA_SQFT,
    h.AUTO_FIRE_NOTIF, h.HOME_SECURITY_SYS, h.SWIMMING_POOL, h.BASEMENT;

```

```

CREATE VIEW vw_auto_policies AS
SELECT
    ap.APOLICY_ID AS policy_ref,
    ap.CUST_ID AS customer_ref,
    c.FIRST_NAME,
    c.LAST_NAME,
    ap.APOLICY_START_DT AS start_dt,
    ap.APOLICY_END_DT AS end_dt,
    ap.APREMIUM_AMT AS premium,
    ap.APOLICY_STATUS AS status_code,
    COUNT(v.VEHICLE_ID) AS vehicle_count
FROM RAH_AUTO_POLICY ap
JOIN RAH_CUSTOMER c ON ap.CUST_ID = c.CUST_ID AND ap.CUST_TYPE = c.CUST_TYPE
LEFT JOIN RAH_VEHICLE v ON ap.APOLICY_ID = v.APOLICY_ID
GROUP BY ap.APOLICY_ID, ap.CUST_ID, c.FIRST_NAME, c.LAST_NAME,
    ap.APOLICY_START_DT, ap.APOLICY_END_DT, ap.APREMIUM_AMT, ap.APOLICY_STATUS;

```

```

CREATE VIEW vw_home_invoices AS
SELECT
    hi.HINVOICE_ID AS invoice_ref,
    hi.HPOLICY_ID AS policy_ref,
    hp.CUST_ID AS customer_ref,

```

```

c.FIRST_NAME,
c.LAST_NAME,
hi.HINVOICE_DT AS invoice_dt,
hi.HINVOICE_DUE_DT AS due_dt,
hi.HINVOICE_AMT AS amount_due,
hp.HPOLICY_STATUS AS status_code,
COALESCE(SUM(hpay.HPAYMENT_AMT), 0) AS paid_amount
FROM RAH_HOME_INVOICE hi
JOIN RAH_HOME_POLICY hp ON hi.HPOLICY_ID = hp.HPOLICY_ID
JOIN RAH_CUSTOMER c ON hp.CUST_ID = c.CUST_ID AND hp.CUST_TYPE = c.CUST_TYPE
LEFT JOIN RAH_HOME_PAYMENT hpay ON hi.HINVOICE_ID = hpay.HINVOICE_ID
GROUP BY hi.HINVOICE_ID, hi.HPOLICY_ID, hp.CUST_ID, c.FIRST_NAME, c.LAST_NAME,
hi.HINVOICE_DT, hi.HINVOICE_DUE_DT, hi.HINVOICE_AMT, hp.HPOLICY_STATUS;

CREATE VIEW vw_auto_invoices AS
SELECT
ai.AINVOICE_ID AS invoice_ref,
ai.APOLICY_ID AS policy_ref,
ap.CUST_ID AS customer_ref,
c.FIRST_NAME,
c.LAST_NAME,
ai.AINVOICE_DT AS invoice_dt,
ai.AINVOICE_DUE_DT AS due_dt,
ai.AINVOICE_AMT AS amount_due,
ap.APOLICY_STATUS AS status_code,
COALESCE(SUM(apay.APAYMENT_AMT), 0) AS paid_amount
FROM RAH_AUTO_INVOICE ai
JOIN RAH_AUTO_POLICY ap ON ai.APOLICY_ID = ap.APOLICY_ID
JOIN RAH_CUSTOMER c ON ap.CUST_ID = c.CUST_ID AND ap.CUST_TYPE = c.CUST_TYPE
LEFT JOIN RAH_AUTO_PAYMENT apay ON ai.AINVOICE_ID = apay.AINVOICE_ID
GROUP BY ai.AINVOICE_ID, ai.APOLICY_ID, ap.CUST_ID, c.FIRST_NAME, c.LAST_NAME,
ai.AINVOICE_DT, ai.AINVOICE_DUE_DT, ai.AINVOICE_AMT, ap.APOLICY_STATUS;

CREATE VIEW vw_all_payments AS
SELECT
'Home' AS kind,
hp.HPAYMENT_ID AS payment_ref,
hp.HPAYMENT_DT AS pay_date,
hp.HPAYMENT_AMT AS amount,
hp.HPAYMENT_METHOD AS method,
hp.HINVOICE_ID AS invoice_ref,
hpol.CUST_ID AS customer_ref,
c.FIRST_NAME,
c.LAST_NAME
FROM RAH_HOME_PAYMENT hp
JOIN RAH_HOME_INVOICE hi ON hp.HINVOICE_ID = hi.HINVOICE_ID
JOIN RAH_HOME_POLICY hpol ON hi.HPOLICY_ID = hpol.HPOLICY_ID
JOIN RAH_CUSTOMER c ON hpol.CUST_ID = c.CUST_ID AND hpol.CUST_TYPE = c.CUST_TYPE
UNION ALL
SELECT
'Auto',
ap.APAYMENT_ID,
ap.APAYMENT_DT,
ap.APAYMENT_AMT,
ap.APAYMENT_METHOD,
ap.AINVOICE_ID,
apol.CUST_ID,
c.FIRST_NAME,
c.LAST_NAME
FROM RAH_AUTO_PAYMENT ap
JOIN RAH_AUTO_INVOICE ai ON ap.AINVOICE_ID = ai.AINVOICE_ID
JOIN RAH_AUTO_POLICY apol ON ai.APOLICY_ID = apol.APOLICY_ID
JOIN RAH_CUSTOMER c ON apol.CUST_ID = c.CUST_ID AND apol.CUST_TYPE = c.CUST_TYPE;

CREATE VIEW vw_customer_dashboard AS
SELECT
c.CUST_ID AS customer_ref,
(
SELECT COUNT(*) FROM RAH_HOME_POLICY hp
WHERE hp.CUST_ID = c.CUST_ID AND hp.HPOLICY_STATUS = 'C'
) + (
SELECT COUNT(*) FROM RAH_AUTO_POLICY ap
WHERE ap.CUST_ID = c.CUST_ID AND ap.APOLICY_STATUS = 'C'
) AS active_policies,
fn_total_premium(c.CUST_ID) AS total_premium,
fn_outstanding_balance(c.CUST_ID) AS outstanding_balance,
(
SELECT COALESCE(SUM(hpay.HPAYMENT_AMT), 0)
FROM RAH_HOME_PAYMENT hpay
JOIN RAH_HOME_INVOICE hi ON hpay.HINVOICE_ID = hi.HINVOICE_ID
JOIN RAH_HOME_POLICY hp ON hi.HPOLICY_ID = hp.HPOLICY_ID
WHERE hp.CUST_ID = c.CUST_ID
) + (

```

```

        SELECT COALESCE(SUM(apay.APAYMENT_AMT), 0)
        FROM RAH_AUTO_PAYMENT apay
        JOIN RAH_AUTO_INVOICE ai ON apay.AINVOICE_ID = ai.AINVOICE_ID
        JOIN RAH_AUTO_POLICY ap ON ai.APOLICY_ID = ap.APOLICY_ID
        WHERE ap.CUST_ID = c.CUST_ID
    ) AS total_paid
FROM RAH_CUSTOMER c;

CREATE VIEW vw_customer_vehicles AS
SELECT
    v.VEHICLE_ID AS vehicle_ref,
    v.VEHICLE_VIN,
    v.VEHICLE_MAKE,
    v.VEHICLE_MODEL,
    v.VEHICLE_YEAR,
    v.VEHICLE_STATUS,
    v.APOLICY_ID AS policy_ref,
    ap.APOLICY_STATUS AS status_code,
    ap.CUST_ID AS customer_ref,
    GROUP_CONCAT(CONCAT(d.DRIVER_FNAME, ' ', d.DRIVER_LNAME, ' (License: ', d.DRIVER_LICENSE_NO, ')') SEPARATOR ';') AS drivers
FROM RAH_VEHICLE v
JOIN RAH_AUTO_POLICY ap ON v.APOLICY_ID = ap.APOLICY_ID
LEFT JOIN RAH_DRIVER d ON v.VEHICLE_ID = d.VEHICLE_ID
GROUP BY v.VEHICLE_ID, v.VEHICLE_VIN, v.VEHICLE_MAKE, v.VEHICLE_MODEL,
         v.VEHICLE_YEAR, v.VEHICLE_STATUS, v.APOLICY_ID, ap.APOLICY_STATUS, ap.CUST_ID;

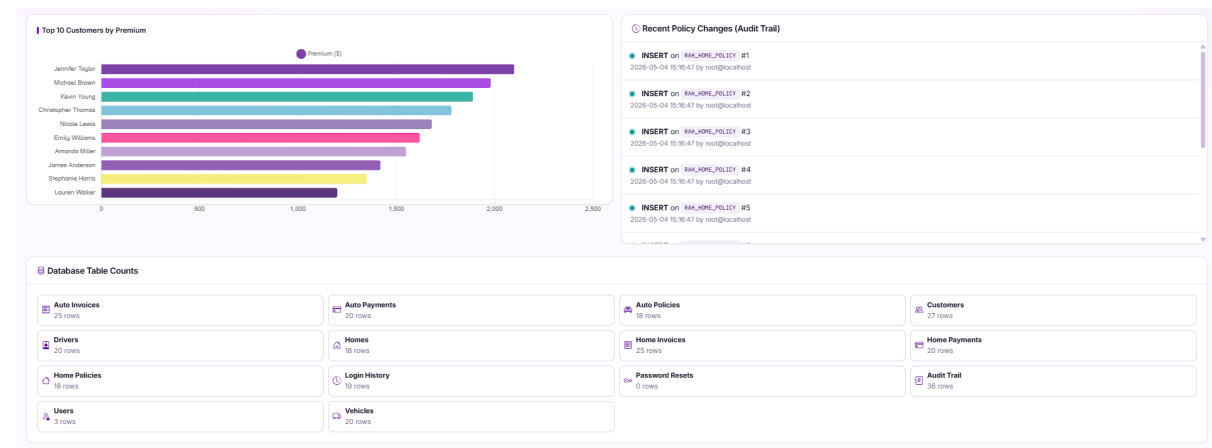
CREATE VIEW vw_employee_vehicles AS
SELECT
    v.VEHICLE_ID AS vehicle_ref,
    v.VEHICLE_VIN,
    v.VEHICLE_MAKE,
    v.VEHICLE_MODEL,
    v.VEHICLE_YEAR,
    v.VEHICLE_STATUS,
    v.APOLICY_ID AS policy_ref,
    ap.CUST_ID AS customer_ref,
    c.FIRST_NAME,
    c.LAST_NAME,
    COUNT(d.DRIVER_ID) AS driver_count
FROM RAH_VEHICLE v
JOIN RAH_AUTO_POLICY ap ON v.APOLICY_ID = ap.APOLICY_ID
JOIN RAH_CUSTOMER c ON ap.CUST_ID = c.CUST_ID AND ap.CUST_TYPE = c.CUST_TYPE
LEFT JOIN RAH_DRIVER d ON v.VEHICLE_ID = d.VEHICLE_ID
GROUP BY v.VEHICLE_ID, v.VEHICLE_VIN, v.VEHICLE_MAKE, v.VEHICLE_MODEL,
         v.VEHICLE_YEAR, v.VEHICLE_STATUS, v.APOLICY_ID, ap.CUST_ID, c.FIRST_NAME, c.LAST_NAME;

CREATE VIEW vw_employee_drivers AS
SELECT
    d.DRIVER_ID AS driver_ref,
    d.DRIVER_LICENSE_NO,
    d.DRIVER_FNAME,
    d.DRIVER_LNAME,
    d.DRIVER_AGE,
    d.VEHICLE_ID AS vehicle_ref,
    CONCAT(v.VEHICLE_YEAR, ' ', v.VEHICLE_MAKE, ' ', v.VEHICLE_MODEL) AS vehicle_label,
    v.VEHICLE_VIN,
    COUNT(*) AS row_marker
FROM RAH_DRIVER d
JOIN RAH_VEHICLE v ON d.VEHICLE_ID = v.VEHICLE_ID
GROUP BY d.DRIVER_ID, d.DRIVER_LICENSE_NO, d.DRIVER_FNAME, d.DRIVER_LNAME,
         d.DRIVER_AGE, d.VEHICLE_ID, v.VEHICLE_YEAR, v.VEHICLE_MAKE, v.VEHICLE_MODEL, v.VEHICLE_VIN;

CREATE VIEW vw_audit_trail AS
SELECT
    AUDIT_ID AS audit_ref,
    TABLE_NAME AS entity,
    RECORD_ID AS ref,
    ACTION AS action,
    CHANGED_BY AS actor,
    CHANGED_AT AS changed_at,
    COUNT(*) AS row_marker
FROM RAH_POLICY_AUDIT
GROUP BY AUDIT_ID, TABLE_NAME, RECORD_ID, ACTION, CHANGED_BY, CHANGED_AT;

-- =====
-- END OF SETUP
-- =====

```



Employee Vehicles Page

NICE Insurance

Dashboard

Manage

Reports

Index Analysis

employee1EMPLOYEE

Manage Vehicles

Customers

Policies

Invoices

Payments

Vehicles

Drivers

VIN, make, model, year, policy #, ...

Search

Clear

ID	VIN	MAKE	MODEL	YEAR	STATUS	POLICY	OWNER	ACTIONS
10	1S1YY2Z6P656789	Chevrolet	Corvette	2024	Owned	AP-9	Daniel Wilson	
13	5AL8523Y56490123	Land Rover	Range Rover	2024	Owned	AP-11	Ashley Jackson	
19	WPBA32A76CL456789	Porsche	911	2024	Owned	AP-17	James Anderson	
6	WDDZF4H81CA765432	Mercedes	C300	2023	Owned	AP-5	Robert Johnson	
20	19K1C2F52NE56789	Honda	Accord	2023	Leased	AP-18	Michael Brown	
3	5Y3SA10N4DP14736	Tesla	Model S	2023	Owned	AP-2	James Anderson	
11	WAUZZZ4G48N678901	Audi	A4	2023	Leased	AP-10	Jennifer Taylor	
17	5Y3E1EABLP234567	Tesla	Model 3	2023	Owned	AP-15	Kevin Young	
18	1F8K8DHL6345678	Ford	Explorer	2022	Leased	AP-16	Rachel King	
4	WBAS45C31P256789	BMW	328i	2022	Leased	AP-3	Sarah Martinez	
9	JTDNA30U56D456789	Toyota	Prius	2022	Leased	AP-8	Amanda Miller	
14	4T1B11H23U912324	Toyota	Camry	2022	Leased	AP-12	Stephanie Harris	
7	3N1BNT3828012345	Nissan	Altima	2021	Leased	AP-6	Michael Brown	

Employee Index Analysis Page

NICE Insurance

Dashboard

Manage

Reports

Index Analysis

employee1EMPLOYEE

Database Index Analysis

Extra Credit

About Index Analysis: This page demonstrates strategic database indexing for performance optimization. Each index below was created to speed up common queries used throughout the application. EXPLAIN output shows how MySQL uses these indexes to optimize query execution.

Custom Indexes Created (18 total columns indexed)

INDEX NAME	TABLE	COLUMN
idx_auto_invoice_policy	rah_auto_invoice	APOLICY_ID
idx_auto_policy_cust	rah_auto_policy	CUST_ID
idx_auto_policy_cust	rah_auto_policy	CUST_TYPE
idx_auto_policy_status	rah_auto_policy	APOLICY_STATUS
idx_customer_city	rah_customer	CITY
idx_customer_state	rah_customer	STATE
idx_customer_type	rah_customer	CUST_TYPE
idx_driver_vehicle	rah_driver	VEHICLE_ID
idx_home_invoice_policy	rah_home_invoice	HPOLICY_ID
idx_home_policy_cust	rah_home_policy	CUST_ID
idx_home_policy_cust	rah_home_policy	CUST_TYPE
idx_home_policy_status	rah_home_policy	HPOLICY_STATUS
idx_login_history_user	rah_login_history	USER_ID
idx_audit_table_record	rah_policy_audit	TABLE_NAME
idx_audit_table_record	rah_policy_audit	RECORD_ID

Users by role

Table: RAH_USER | Columns: ROLE

Rationale: Role-based query optimization for authentication and access control

EXTRA	FILTERED	ID	KEY	KEY_LEN	PARTITIONS	POSSIBLE_KEYS	REF	ROWS	SELECT_TYPE	TABLE	TYPE
NULL	100	1	idx_user_role	1	NULL	idx_user_role	const	2	SIMPLE	RAH_USER	ref

Customer lookup by city

Table: RAH_CUSTOMER | Columns: CITY

Rationale: Frequent filtering by city for geographic analytics

EXTRA	FILTERED	ID	KEY	KEY_LEN	PARTITIONS	POSSIBLE_KEYS	REF	ROWS	SELECT_TYPE	TABLE	TYPE
NULL	100	1	idx_customer_city	202	NULL	idx_customer_city	const	2	SIMPLE	RAH_CUSTOMER	ref

Index Design Rationale

idx_customer_state / idx_customer_city

Customers are frequently filtered by geographic location for analytics and reports. These indexes turn full table scans into efficient index lookups.

idx_home_invoice_policy / idx_auto_invoice_policy

Invoice pages join invoices to policies. These foreign key indexes optimize the JOIN and also speed up cascading deletes when a policy is removed.

idx_home_policy_cust / idx_auto_policy_cust

Every customer dashboard and detail page joins policies to customers. These indexes dramatically speed up the JOIN operation on CUST_ID.

idx_vehicle_policy

Vehicle listings are always filtered by auto policy. This index enables efficient lookups of vehicles belonging to a specific policy.

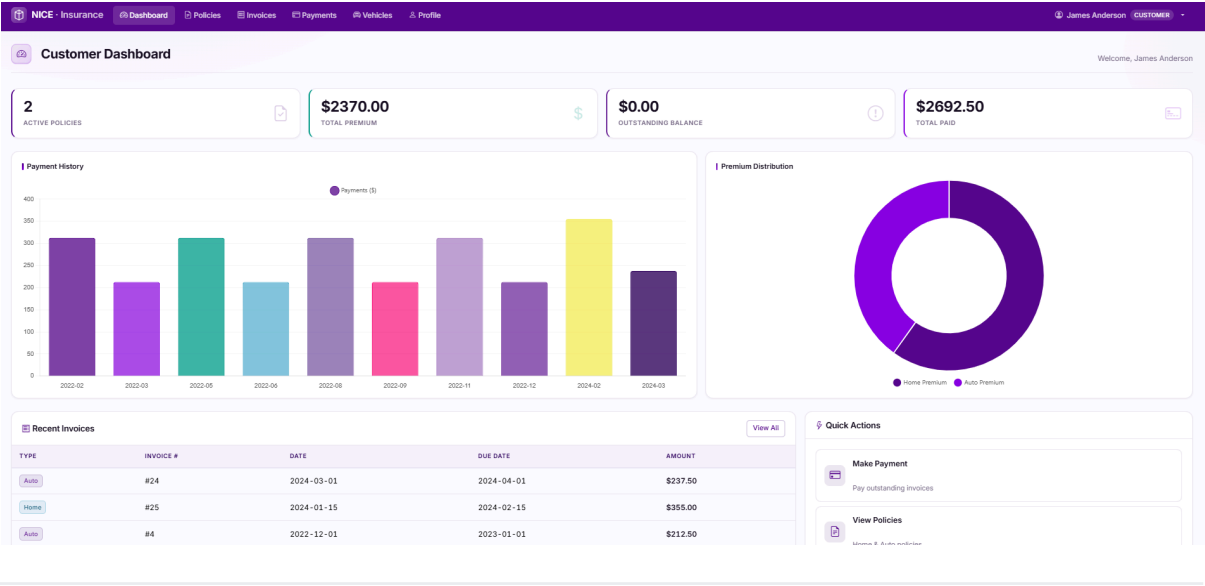
idx_home_policy_status / idx_auto_policy_status

Dashboard stats and policy listings frequently filter by active/expired status. With only 2 distinct values, the optimizer uses these efficiently for partial-table reads.

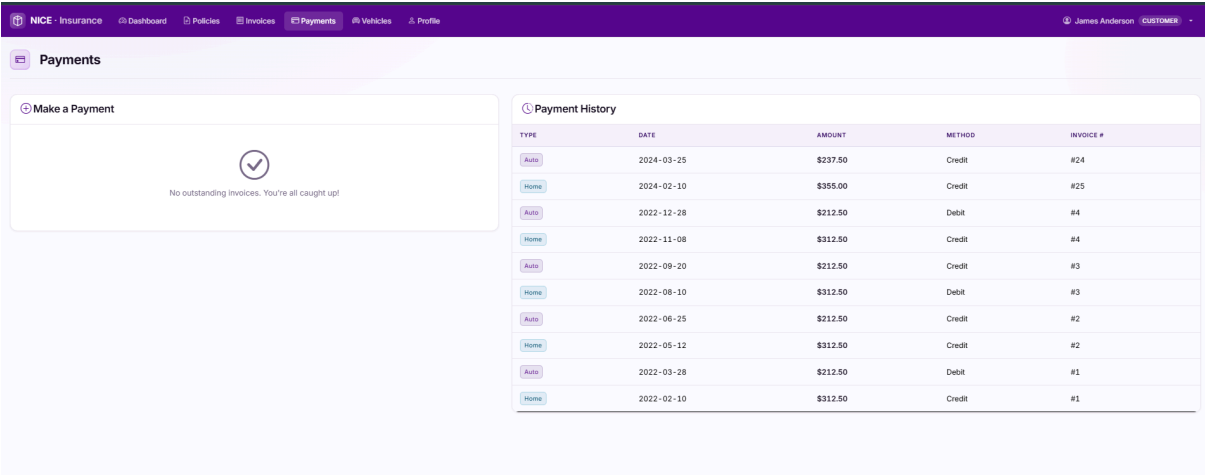
idx_user_role / idx_user_custid

Authentication and authorization queries filter users by role and customer ID on every request. These indexes keep login and session validation fast.

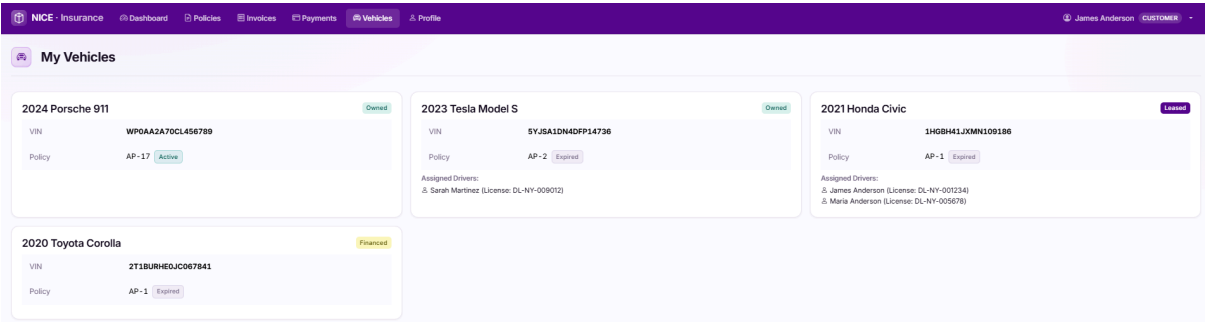
Customer Dashboard



Customer Make Payment Page



Customer Vehicles Page



Customer Profile Page

Dashboard

Policies

Invoices

Payments

Vehicles

Profile

James AndersonCUSTOMER

My Profile

James Michael Anderson

Customer #1

HomeAuto

Username: customer1

Email: james.anderson@email.com

Member since: May 04, 2026

Last login: May 06, 2026 02:40 PM

Gender: Male

Marital Status: Married

Update Address

Address Line 1

142 Oak Street

Address Line 2

Apt 3B

City

Brooklyn

State

NY

ZIP Code

11201

Save Changes

Account Lockout Message (5 Attempts)

Invalid password. 1 attempt(s) remaining.

WELCOME BACK

Sign in to your account

Enter your credentials to continue.

Username

e.g. customer1

Password

Enter password

Forgot password?

Sign in →

Account has been locked due to too many failed attempts.

WELCOME BACK

Sign in to your account

Enter your credentials to continue.

Username

e.g. customer1

Password

Enter password

Forgot password?

Sign in →

Password Reset Flow (Security Question + New Password)

ACCOUNT RECOVERY

Reset your password

Verify your identity using your security question.

Username

customer2

Lookup

Security question

What city were you born in?

Your answer

New password

Enter new password

Reset password

[← Back to sign in](#)

Section 6: Features

Customer Portal Features

- Secure registration and login: Customers can create accounts, log in safely, and access their insurance information through authenticated sessions.
- Dashboard summary: The customer dashboard provides a quick view of policies, payments, balances, and recent account activity.
- Home and auto policy access: Customers can view details of their home and auto insurance policies in one centralized portal.
- Invoice and payment status tracking: The system allows customers to review invoices and check whether payments are paid, partial, or outstanding.
- Online payments and payment history: Customers can make payments through the portal and track their full payment history over time.
- Vehicles and assigned drivers: Auto insurance users can view insured vehicles along with the drivers associated with them.
- Profile and address updates: Customers can update personal and address details directly without employee assistance.

Employee Portal Features

- Customer and profile management: Employees can manage customer records and maintain accurate customer profile information.
- Policy lifecycle management: Employees can create, update, delete, and renew both home and auto insurance policies.
- Invoice and payment operations: Employees can generate invoices and record or monitor customer payments within the system.
- Vehicle and driver management: The platform allows employees to manage insured vehicles and assign or update driver information.
- Dashboards and business reports: Employees can access dashboards and analytical reports for operational and financial monitoring.
- Audit trail access: Employees can review audit logs to track important policy-related changes made in the system.
- Search and query analysis: The portal supports searching across system data and analyzing query performance for reporting and optimization.

Additional Features

- Role-based access control: The application separates customer and employee permissions so each user only accesses the functions relevant to their role.
- Read-only security views: The system uses database views to provide controlled, simplified, and secure access to frequently used business data.
- Stored procedure-based operations: Major operations such as registration, payments, renewals, invoice generation, and record updates are handled through stored procedures for better consistency.
- Parameterized query execution: User input is processed through parameterized SQL queries to reduce the risk of SQL injection and improve query safety.
- Password security with bcrypt: User passwords and security answers are stored as hashed values instead of plain text for stronger authentication security.
- CSRF-protected form submissions: Forms include CSRF protection to prevent unauthorized or forged requests.
- Account lockout and login tracking: The system records login attempts and locks accounts after repeated failures to strengthen security.
- Password reset support: Users can reset passwords through a security-question-based recovery workflow.
- Dashboard analytics and charts: The application includes visual reporting through charts for revenue, payments, policy distribution, and other business metrics.
- Audit logging: Policy-related changes are recorded in audit tables so administrative actions can be tracked and reviewed.
- Index analysis support: The employee side includes index and query analysis features to study database performance and optimization.
- Customer self-service functionality: Customers can independently manage account information, review billing, and track insurance activity without relying on staff.
- Business workflow integration: Policies, invoices, payments, vehicles, and drivers are all connected in one database-driven workflow rather than isolated modules.
- Session timeout management: User sessions automatically expire after inactivity to reduce unauthorized access risk.
- Data integrity through constraints and triggers: The database uses keys, checks, triggers, and relationships to enforce consistency across records.

Section 7: Extra Features

The NICE Insurance Management System includes several advanced features implemented beyond the basic project requirements to improve application scalability, performance, security, and user experience. These features demonstrate the practical integration of modern database concepts, secure web application development, and business analytics within the system.

7.1 Docker Containerization

The application was containerized using Docker to provide a consistent and portable deployment environment. Docker containers were used for both the Flask web application and the MySQL database, while Docker Compose managed multi-container orchestration and communication between services. This approach simplified dependency management, reduced environment configuration issues, and improved deployment efficiency across different systems. Containerization also improves scalability and maintainability of the application architecture.

7.2 Database Indexing and Performance Optimization

Indexes were implemented on frequently accessed columns to improve query execution performance and optimize database operations. Proper indexing strategies were applied on attributes commonly used in JOIN operations, filtering conditions, and authentication queries such as customer identifiers, policy identifiers, usernames, invoice references, and payment records. Query performance and execution plans were analyzed using MySQL EXPLAIN statements to evaluate how indexes improved data retrieval efficiency and reduced query execution cost.

7.3 Data Visualization and Analytics

Interactive charts and dashboards were implemented to provide visual analysis of insurance-related business data. The application includes graphical representations for payment trends, policy distribution, outstanding balances, revenue analysis, and customer statistics. These visualizations help employees analyze operational performance and improve decision-making by presenting complex business data in an understandable graphical format.

7.4 Advanced Security Features

The NICE Insurance web application implements multiple layers of security to protect user data, prevent common web vulnerabilities, and maintain an audit trail for compliance and monitoring purposes.

Password Encryption

All user passwords are encrypted using the bcrypt hashing algorithm before being stored in the database. The application stores only hashed password values and never stores or logs plain-text passwords. During authentication, submitted passwords are securely compared against the stored bcrypt hash.

SQL Injection Prevention

Parameterized queries and prepared statements were used throughout the application to prevent SQL injection attacks. User inputs are passed as query parameters rather than directly concatenated into SQL statements. Stored procedures were also utilized for secure database operations and controlled execution of business logic.

Cross-Site Scripting (XSS) Prevention

User-generated content displayed within the application is sanitized before rendering in HTML templates. Flask and Jinja2 autoescaping features were used to prevent malicious scripts from being injected through forms or URL parameters.

CSRF Protection

Cross-Site Request Forgery protection was implemented by generating unique CSRF tokens for user sessions and validating these tokens during form submissions and POST requests. AJAX requests also include CSRF validation headers to prevent unauthorized requests.

Account Lockout Mechanism

The system automatically locks user accounts after multiple consecutive failed login attempts. Failed login counters are maintained within the database, and locked accounts cannot authenticate until administrative action is taken. This feature helps reduce brute-force login attacks.

Login History Tracking

All login attempts, including successful and failed authentication attempts, are recorded in the login history table with timestamps, IP addresses, and login status information. This provides an audit trail for monitoring suspicious activities and user access patterns.

Session Management

Secure server-side session management was implemented using Flask sessions and secret keys. User sessions automatically expire after a period of inactivity to reduce the risk of unauthorized access from unattended sessions.

Password Reset Security

Users can securely reset passwords using predefined security questions and verification mechanisms. Password reset requests are tracked with expiration timestamps to prevent unauthorized reuse of reset credentials.

Role-Based Access Control

The application enforces strict role-based authorization using separate customer and employee access controls. Customers can access only their own records and insurance information, while employees have permission to access administrative and management functionalities. Unauthorized access attempts are automatically restricted and redirected to the login interface.

Section 8: Business Analysis with 6 SQL Queries

Below are 6 new queries that satisfy each required category.

Q1. Table joins with at least 3 tables

A1) Select query

```
```sql
```

```
SELECT
 c.CUST_ID AS customer_id,
 CONCAT(c.FIRST_NAME, ' ', c.LAST_NAME) AS customer_name,
 ap.APOLICY_ID AS auto_policy_id,
 COUNT(DISTINCT v.VEHICLE_ID) AS total_vehicles,
 COUNT(DISTINCT d.DRIVER_ID) AS total_drivers
FROM RAH_CUSTOMER c
JOIN RAH_AUTO_POLICY ap
 ON c.CUST_ID = ap.CUST_ID
 AND c.CUST_TYPE = ap.CUST_TYPE
LEFT JOIN RAH_VEHICLE v
 ON ap.APOLICY_ID = v.APOLICY_ID
LEFT JOIN RAH_DRIVER d
 ON v.VEHICLE_ID = d.VEHICLE_ID
GROUP BY c.CUST_ID, CONCAT(c.FIRST_NAME, ' ', c.LAST_NAME), ap.APOLICY_ID
ORDER BY total_drivers DESC, total_vehicles DESC, customer_id ASC;
```
```

A2) Result of the query

Returns one row per auto policy with the customer name, policy ID, total insured vehicles, and total assigned drivers.

| | customer_id | customer_name | auto_policy_id | total_vehicles | total_drivers |
|---|-------------|------------------|----------------|----------------|---------------|
| ▶ | 21 | James Anderson | 1 | 2 | 2 |
| | 25 | Jennifer Taylor | 10 | 2 | 2 |
| | 15 | Andrew Clark | 13 | 1 | 2 |
| | 22 | Sarah Martinez | 3 | 1 | 2 |
| | 3 | Robert Johnson | 5 | 1 | 1 |
| | 5 | Michael Brown | 6 | 1 | 1 |
| | 5 | Michael Brown | 18 | 1 | 1 |
| | 7 | William Garcia | 7 | 1 | 1 |
| | 9 | Daniel Wilson | 9 | 1 | 1 |
| | 12 | Ashley Jackson | 11 | 1 | 1 |
| | 17 | Joshua Robinson | 14 | 1 | 1 |
| | 20 | Rachel King | 16 | 1 | 1 |
| | 21 | James Anderson | 2 | 1 | 1 |
| | 24 | Amanda Miller | 8 | 1 | 1 |
| | 26 | Stephanie Harris | 12 | 1 | 1 |
| | 27 | Kevin Young | 15 | 1 | 1 |
| | 3 | Robert Johnson | 4 | 1 | 0 |
| | 21 | James Anderson | 17 | 1 | 0 |

A3) Business Purpose

This query answers: Which auto policy customers have the largest number of insured vehicles and assigned drivers?

Q2. Multi-row subquery

A1) Select query

```
```sql
```

```
SELECT
 customer_ref AS customer_id,
 CONCAT(FIRST_NAME, ' ', LAST_NAME) AS customer_name,
 ROUND(fn_total_premium(customer_ref), 2) AS total_active_premium
FROM vw_customer_directory
WHERE customer_ref IN (
 SELECT CUST_ID
 FROM RAH_HOME_POLICY
 WHERE HPOLICY_STATUS = 'C'
 UNION
 SELECT CUST_ID
 FROM RAH_AUTO_POLICY
 WHERE APOLICY_STATUS = 'C'
)
ORDER BY total_active_premium DESC, customer_id ASC;
```
```

A2) Result of the query

Returns customers who currently hold at least one active policy, along with their total active premium amount.

| | customer_id | customer_name | total_active_premium |
|---|-------------|--------------------|----------------------|
| ▶ | 10 | Jennifer Taylor | 2100.00 |
| | 5 | Michael Brown | 1980.00 |
| | 19 | Kevin Young | 1890.00 |
| | 11 | Christopher Thomas | 1780.00 |
| | 16 | Nicole Lewis | 1680.00 |
| | 4 | Emily Williams | 1620.00 |
| | 8 | Amanda Miller | 1550.00 |
| | 1 | James Anderson | 1420.00 |
| | 14 | Stephanie Harris | 1350.00 |
| | 18 | Lauren Walker | 1200.00 |
| | 25 | Jennifer Taylor | 1200.00 |
| | 27 | Kevin Young | 1150.00 |
| | 6 | Jessica Davis | 1100.00 |
| | 2 | Sarah Martinez | 1050.00 |
| | 26 | Stephanie Harris | 1050.00 |
| | 7 | William Garcia | 990.00 |
| | 21 | James Anderson | 950.00 |
| | 17 | Joshua Robinson | 940.00 |
| | 13 | Matthew White | 920.00 |
| | 23 | Michael Brown | 890.00 |
| | 24 | Amanda Miller | 860.00 |
| | 15 | Andrew Clark | 820.00 |
| | 9 | Daniel Wilson | 780.00 |
| | 22 | Sarah Martinez | 750.00 |
| | 3 | Robert Johnson | 730.00 |
| | 20 | Rachel King | 710.00 |
| | 12 | Ashley Jackson | 650.00 |

A3) Business Purpose

This query answers: Which currently active customers generate the most premium revenue for the company?

Q3. Correlated subquery

A1) Select query

```
```sql
```

```
SELECT
 c.CUST_ID AS customer_id,
 CONCAT(c.FIRST_NAME, ' ', c.LAST_NAME) AS customer_name,
 c.STATE AS customer_state
FROM RAH_CUSTOMER c
WHERE (
 SELECT COUNT(*)
 FROM RAH_HOME_POLICY hp
 WHERE hp.CUST_ID = c.CUST_ID
) >= 2
ORDER BY c.STATE ASC, customer_name ASC;
```
```

A2) Result of the query

Returns customers who have two or more home policies, along with their names and states.

| | customer_id | customer_name | customer_state |
|---|-------------|----------------|----------------|
| ▶ | 8 | Amanda Miller | CT |
| | 4 | Emily Williams | CT |
| | 1 | James Anderson | NY |
| | 2 | Sarah Martinez | NY |

A3) Business Purpose

This query answers: Which customers are repeat or high-value home insurance customers based on multiple home policies?

Q4. SET operator query

A1) Select query

```
```sql
```

```
SELECT
 CUST_ID AS customer_id,
```



```

 'HOME_ONLY' AS policy_segment
FROM RAH_HOME_POLICY
WHERE CUST_ID NOT IN (
 SELECT CUST_ID FROM RAH_AUTO_POLICY
)

UNION

SELECT
 CUST_ID AS customer_id,
 'AUTO_ONLY' AS policy_segment
FROM RAH_AUTO_POLICY
WHERE CUST_ID NOT IN (
 SELECT CUST_ID FROM RAH_HOME_POLICY
)
ORDER BY policy_segment ASC, customer_id ASC;

```

A2) Result of the query

Returns customers who belong exclusively to one line of business: either home-only or auto-only.

Result Grid   Filter Rows:		
	customer_id	policy_segment
▶	3	AUTO_ONLY
	5	AUTO_ONLY
	7	AUTO_ONLY
	9	AUTO_ONLY
	12	AUTO_ONLY
	15	AUTO_ONLY
	17	AUTO_ONLY
	20	AUTO_ONLY
	21	AUTO_ONLY
	22	AUTO_ONLY
	24	AUTO_ONLY
	25	AUTO_ONLY
	26	AUTO_ONLY
	27	AUTO_ONLY
	1	HOME_ONLY
	2	HOME_ONLY
	4	HOME_ONLY
	6	HOME_ONLY
	8	HOME_ONLY
	10	HOME_ONLY
	11	HOME_ONLY
	13	HOME_ONLY
	14	HOME_ONLY
	16	HOME_ONLY
	18	HOME_ONLY
	19	HOME_ONLY
	23	HOME_ONLY

A3) Business Purpose

This query answers: Which customers have only one insurance product, making them good candidates for cross-selling?

### Q5. Query with in-line view / WITH clause

A1) Select query

```sql

```
WITH invoice_summary AS (  
  SELECT  
    'Home' AS policy_type,  
    hp.CUST_ID AS customer_id,  
    SUM(hi.HINVOICE_AMT) AS total_invoiced  
  FROM RAH_HOME_INVOICE hi  
  JOIN RAH_HOME_POLICY hp  
    ON hi.HPOLICY_ID = hp.HPOLICY_ID  
  GROUP BY hp.CUST_ID  
  
  UNION ALL  
  
  SELECT  
    'Auto' AS policy_type,  
    ap.CUST_ID AS customer_id,  
    SUM(ai.AINVOICE_AMT) AS total_invoiced  
  FROM RAH_AUTO_INVOICE ai  
  JOIN RAH_AUTO_POLICY ap  
    ON ai.APOLICY_ID = ap.APOLICY_ID  
  GROUP BY ap.CUST_ID  
)  
SELECT  
  customer_id,  
  ROUND(SUM(total_invoiced), 2) AS total_billed_amount  
FROM invoice_summary  
GROUP BY customer_id  
ORDER BY total_billed_amount DESC, customer_id ASC;  
```
```

A2) Result of the query

Returns each customer's combined billed amount across home and auto invoices.

Result Grid			Filter Rows:
	customer_id	total_billed_amount	
▶	2	1767.50	
	1	1605.00	
	10	1575.00	
	21	1087.50	
	19	945.00	
	16	840.00	
	6	825.00	
	4	810.00	
	5	770.00	
	25	600.00	
	22	562.50	
	3	547.50	
	7	495.00	
	9	390.00	
	26	262.50	
	17	235.00	
	13	230.00	
	15	205.00	
	20	177.50	
	12	162.50	

### A3) Business Purpose

This query answers: Which customers have generated the highest total billed invoice value across all insurance products?

### Q6. TOP-N / BOTTOM-N query

#### A1) Select query

```
```sql
```

```
SELECT
```

```
    hp.HPOLICY_ID AS home_policy_id,
```

```
    hp.CUST_ID AS customer_id,
```

```
    ROUND(hp.HPREMIUM_AMT, 2) AS premium_amount,
```

```
    hp.HPOLICY_STATUS AS policy_status
```

```
FROM RAH_HOME_POLICY hp
```

```
WHERE hp.HPOLICY_STATUS = 'C'
```

```
ORDER BY hp.HPREMIUM_AMT DESC, hp.HPOLICY_ID ASC
```

```
LIMIT 5;
```

```
```
```

#### A2) Result of the query

Returns the top 5 active home policies with the highest premium amounts.

| Result Grid |                | Filter Rows: |                | Export:       | Wrap Ce |
|-------------|----------------|--------------|----------------|---------------|---------|
|             | home_policy_id | customer_id  | premium_amount | policy_status |         |
| ▶           | 11             | 10           | 2100.00        | C             |         |
|             | 17             | 19           | 1890.00        | C             |         |
|             | 12             | 11           | 1780.00        | C             |         |
|             | 15             | 16           | 1680.00        | C             |         |
|             | 6              | 4            | 1620.00        | C             |         |

### A3) Business Purpose

This query answers: Which active home insurance policies contribute the highest premium amounts to the business?

## Section 9: Lessons Learned and Data Sources

---

### What We Learned

---

During the development of the NICE Insurance Management System, the team faced several technical and coordination challenges while designing and implementing a secure full-stack database application. One of the primary challenges was integrating the Flask backend application with the MySQL relational database while maintaining proper session handling, transaction management, and reliable CRUD operations. Managing relationships between multiple interconnected entities such as customers, policies, invoices, vehicles, and payments required careful database normalization and foreign key design to maintain referential integrity. Additional challenges were encountered while implementing stored procedures, triggers, and transaction handling mechanisms to ensure consistency during concurrent database operations.

Security implementation was another important learning experience during the project. Implementing bcrypt password hashing, CSRF protection, parameterized SQL queries, session timeout handling, and role-based access control required careful coordination between frontend forms, backend Flask routes, and database operations. Preventing SQL injection and cross-site scripting vulnerabilities while maintaining application usability also required extensive testing and debugging. The team additionally learned how secure authentication workflows and login monitoring mechanisms can improve overall application security and user protection.

Team coordination and project management also played a major role throughout the development process. Since multiple components such as frontend development, backend APIs, database design, and testing were being developed simultaneously, maintaining consistent communication and version control was necessary to avoid integration conflicts. GitHub was used to manage source code collaboration and synchronize project updates between team members. Through this project, the team gained practical experience in full-stack web development, relational database design, secure application development, and collaborative software engineering practices.

## Conclusion

---

The NICE Insurance Management System successfully demonstrates the design and implementation of a secure, database-driven web application for managing Home and Auto insurance services. The project integrates relational database concepts, full-stack web development, and business process automation into a centralized platform that supports customer management, policy administration, invoice generation, payment processing, vehicle management, and employee operations. The system was developed using Flask, MySQL, Bootstrap, and Docker technologies while following normalization principles, secure coding practices, and multi-tier architecture design.

The project also highlights the practical application of advanced database concepts including stored procedures, triggers, functions, views, transactions, indexing, and referential integrity constraints. Multiple security mechanisms such as bcrypt password hashing, parameterized SQL queries, CSRF protection, session management, and role-based access control were implemented to protect sensitive customer and insurance data. Through this project, the team gained valuable experience in relational database design, secure web application development, backend integration, and collaborative software engineering practices.

Overall, the NICE Insurance Management System provides an efficient, scalable, and secure solution for insurance management operations while demonstrating the successful integration of database management systems concepts with modern web technologies.

## Future Improvements

---

The NICE Insurance Management System provides a strong foundation for managing Home and Auto insurance operations; however, several enhancements can be implemented in the future to further improve scalability, functionality, and user experience. The application can be deployed on cloud platforms such as AWS or Azure to improve availability, scalability, and backup management. Additional security enhancements such as multi-factor authentication (MFA), CAPTCHA validation, and OAuth-based authentication can also be integrated to strengthen account protection. The system can further be expanded with email and SMS notification services for policy renewals, invoice reminders, payment confirmations, and security alerts.

Future versions of the application can also include online claim submission and claim tracking modules to support complete insurance lifecycle management. Integration with third-party payment gateways can enable secure online

transactions and automated payment processing. The analytics capabilities of the application can be enhanced through advanced dashboards, predictive analytics, and business intelligence reporting tools. Additional improvements such as mobile application support, API integration, caching, replication, and support for additional insurance services including health and life insurance can further increase the scalability and business value of the NICE Insurance Management System.

## Data Sources

---

The project code repository is available at: [NICE Insurance Web Application](https://github.com/rigvedrs/NICE_Insurance_App) ([https://github.com/rigvedrs/NICE\\_Insurance\\_App](https://github.com/rigvedrs/NICE_Insurance_App))

The main data source for this project is the SQL setup file included in the repository, which creates the schema, stored procedures, triggers, views, indexes, and sample data used by the application. In other words, the dataset is internally generated as part of the system setup rather than imported from a third-party external data provider.